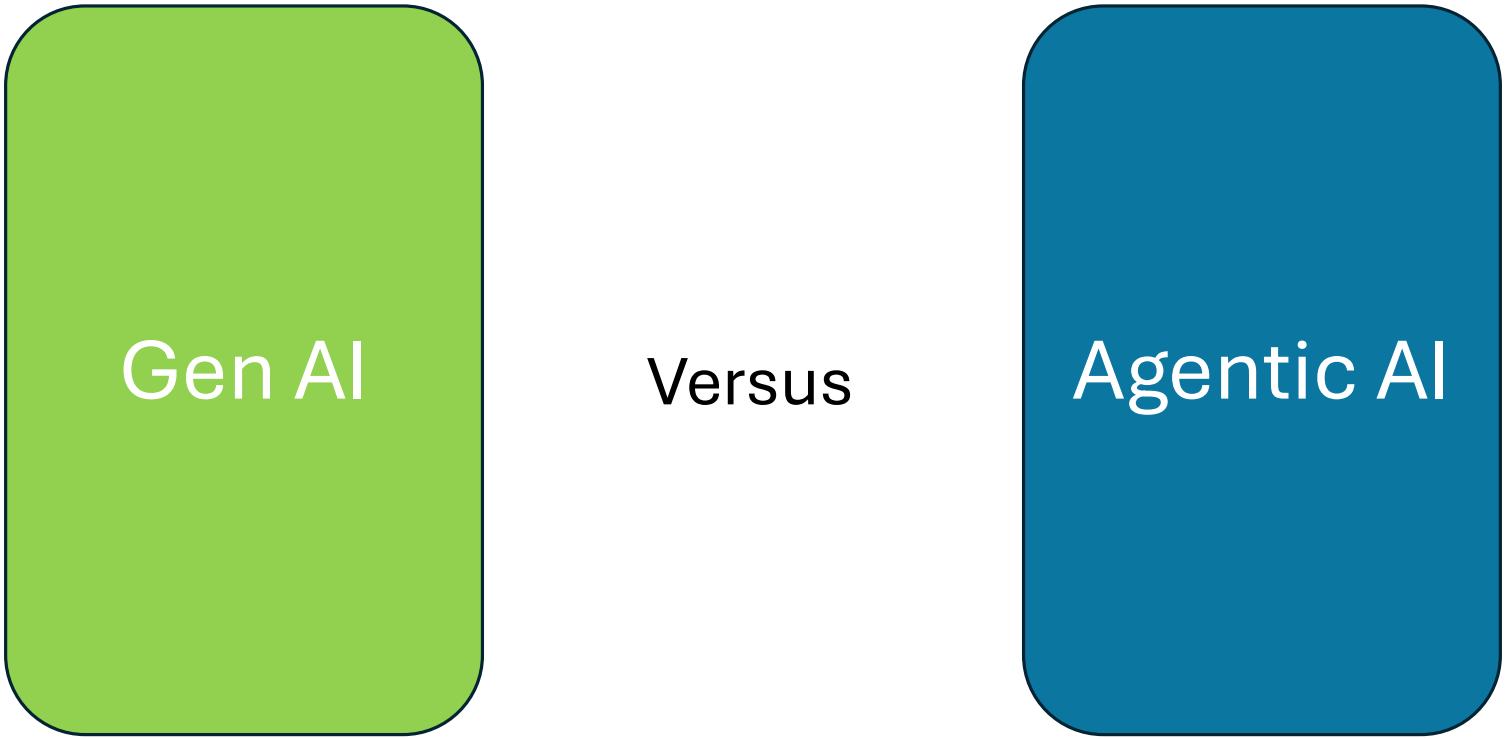

Gen AI
Versus
Agentic AI

You How will be the weather in London next week? I want to plan a trip to London for 3 days

LLM – That's good actually, weather will be very pleasant. Nice time of the year to visit London

You Great, can you look for cheap flights to London from Paris

LLM – Sure, here are the different options sorted by price Flight Information Lookup Operation – RAG / MCP

You Thanks, suggest me some good locations for hotel and suggest me best budget hotels in those areas Lookup Operation – RAG / MCP

LLM – Why not, please find the hotel information as requested.

You Thanks you have been helpful

You – I am planning a trip to London in next 2 weeks. I am fine with any days of the week. Can you look for the best weather and if there is a good weather forecast for continuous 3 days, can you book a cheap flight for me from Paris and also book a hotel in a good tourist location with reasonable price. My budget is maximum 5K USD

AI Agent – Sure, weather is good all the days next week. As you don't mind visiting on week days, based on the flight price and hotel price, it is best to visit starting Tuesday next week and come back by Friday. Here is your flight and hotel details, which are booked for you. Enjoy your time in London. Let me suggest also few places to visit.

How an AI Agent Works

Perceive → Reason → Act → Evaluate

1 Perceive

"What is happening?"



The agent receives information from the user and its environment.

- Understand user request
- Gather relevant context
- Identify key details

2 Reason

"What should I do?"



The agent decides how to accomplish the goal.

- Identify what's needed
- Choose the right tools
- Plan the next steps
- Decide on the best approach

3 Act

"Do it"



The agent performs the required actions using tools and resources.

- Call APIs / search
- Query databases
- Execute code
- Use other agents / services

4 Evaluate

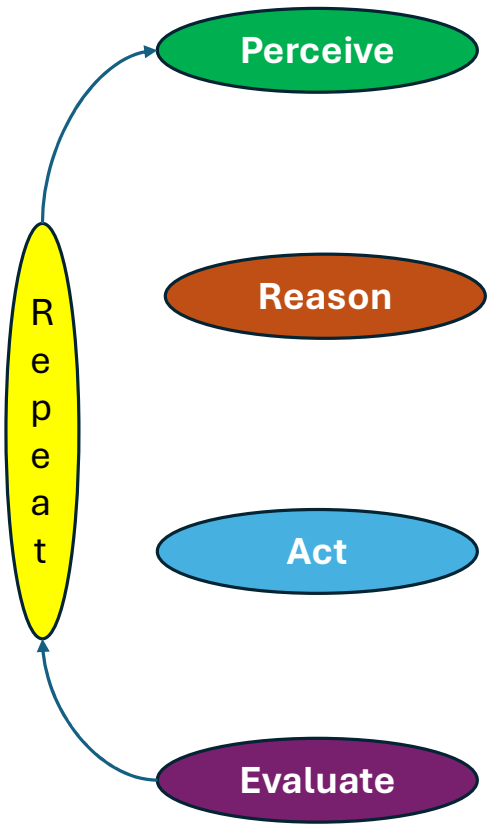
"Did it work?"



The agent checks the results against the goal and decides what to do next.

- Check if the goal is met
- Handle errors or gaps
- Decide to continue or respond

Repeat until goal is achieved



Gathering Data - The agent takes in your prompt, recognizes information and the constraints like budget and location, and checks available tools

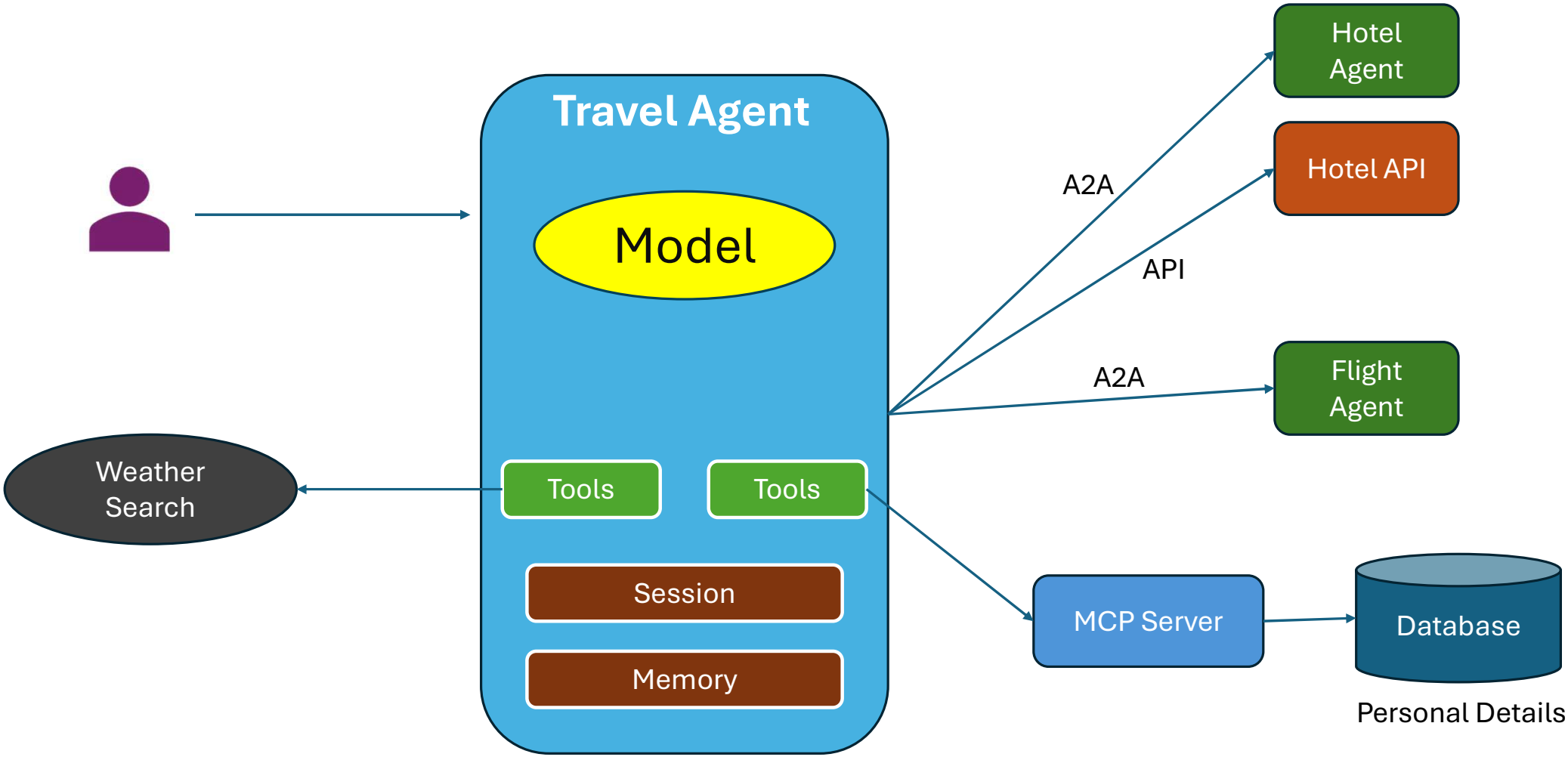
Planning & Decide - The LLM analyzes the inputs and maps out a strategy, thinking: "I need to query a flight Agent / API, a hotel Agent/API, check for specific dates and weather, and a maximum price filter

Executing a Tool - The agent executes the plan by triggering weather search query, flight/hotel aggregator tool or API or Agents

Reviewing Results – Collects all the information and validate against the end goal. If the goal is met, user is notified. If not- suppose the budget exceeds 5k, it loops back to Perceive and Reason to try alternative dates or hotels or flights

You – I am planning a trip to London in next 2 weeks. I am fine with any days of the week. Can you look for the best weather and if there is a good weather forecast for continuous 3 days, can you book a cheap flight for me from Paris and also book a hotel in a good tourist location with reasonable price. My budget is maximum 5K USD

AI Agent – Sure, weather is good all the days next week. As you don't mind visiting on week days, based on the flight price and hotel price, it is best to visit starting Tuesday next week and come back by Friday. Here is your flight and hotel details, which are booked for you. Enjoy your time in London. Let me suggest also few places to visit.


Travel Agent
Model
Tools
Tools
Session
Memory
Weather Search
A2A
API
A2A
Hotel Agent
Hotel API
Flight Agent
MCP Server
Database
Personal Details

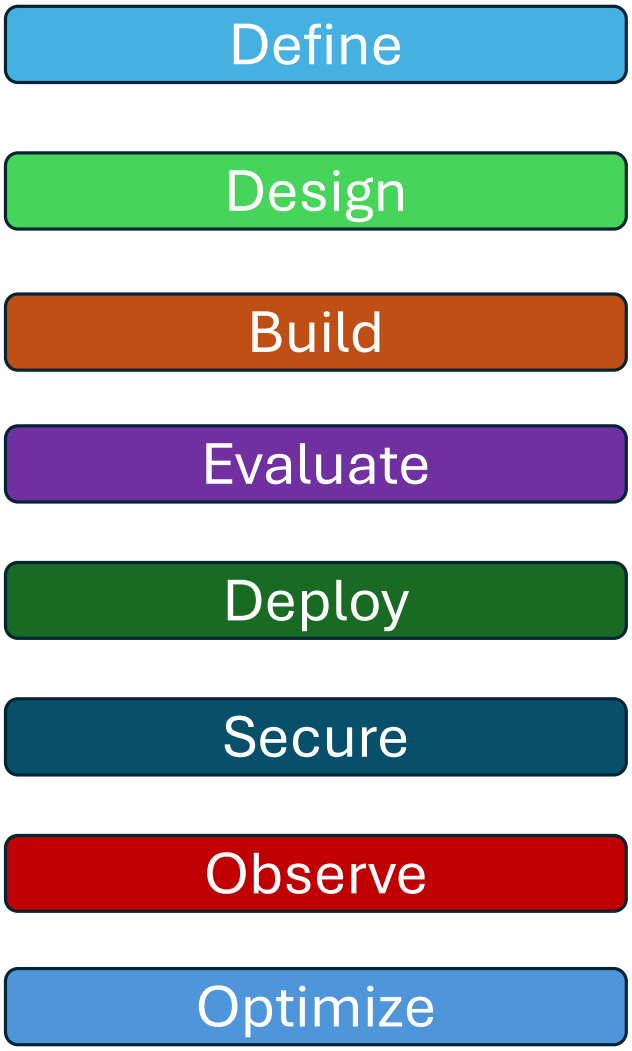
You are an agentic architect. If an use case given, would you always say:

"**Let me Design & Develop an Agent for you?**"

Is this the only choice?

SaaS Agents	**Ready-made agent provided as a software service**
No-Code Agents	**Build an agent through visual/configuration interfaces**
Low-Code Agents	**Build using a framework + some code**
Custom Agents	**Build the entire agent application using code**

	SaaS Agents	No-Code Agents	Low-Code Agents	Custom Agents
Developer effort	Very low	Low	Medium	High
Coding required	Usually none	None	Some	Significant
Control	Limited	Limited–Moderate	Moderate–High	Very High
Customization	Mostly configuration	Configuration + predefined capabilities	Custom logic, tools, workflows	Almost everything can be customized
Tools	Vendor-provided	Prebuilt connectors/tools	Prebuilt + custom tools	Fully custom tools/integrations
Workflow	Usually predefined	Visual workflows/ configuration	Programmatic + visual	Fully programmatic
Data integration	Vendor-supported integrations	Connectors	APIs, connectors, custom integrations	Any integration you can implement
Deployment	Managed by vendor	Usually managed platform	Platform-managed or your infrastructure	You control deployment/runtime
Best for	Common business tasks	Simple/standard business agents	Enterprise agents needing customization	Complex, highly specialized agents



Understand the use case, business goal, success criteria, risk, timeline

Architecture, framework, models, tools, workflow, security

Agent code, tool integration, memory, session, guardrails

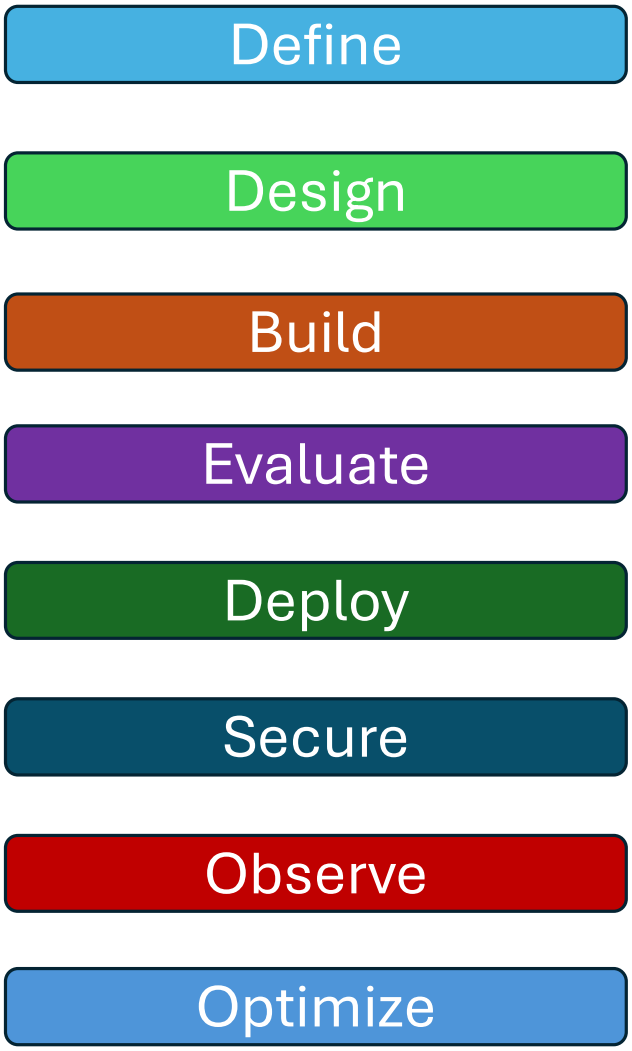
Functional testing, response quality, trajectory, golden dataset, safety

Infrastructure, runtime, scaling, versioning, connectivity

Identity, access control, tool permission, data protection, guardrails

Logs, metrics, traces, latency, token usage, agent quality

Re-evaluation, cost optimization, model selection, re-prompting



Google ADK, Gemini Models, Gemma Models

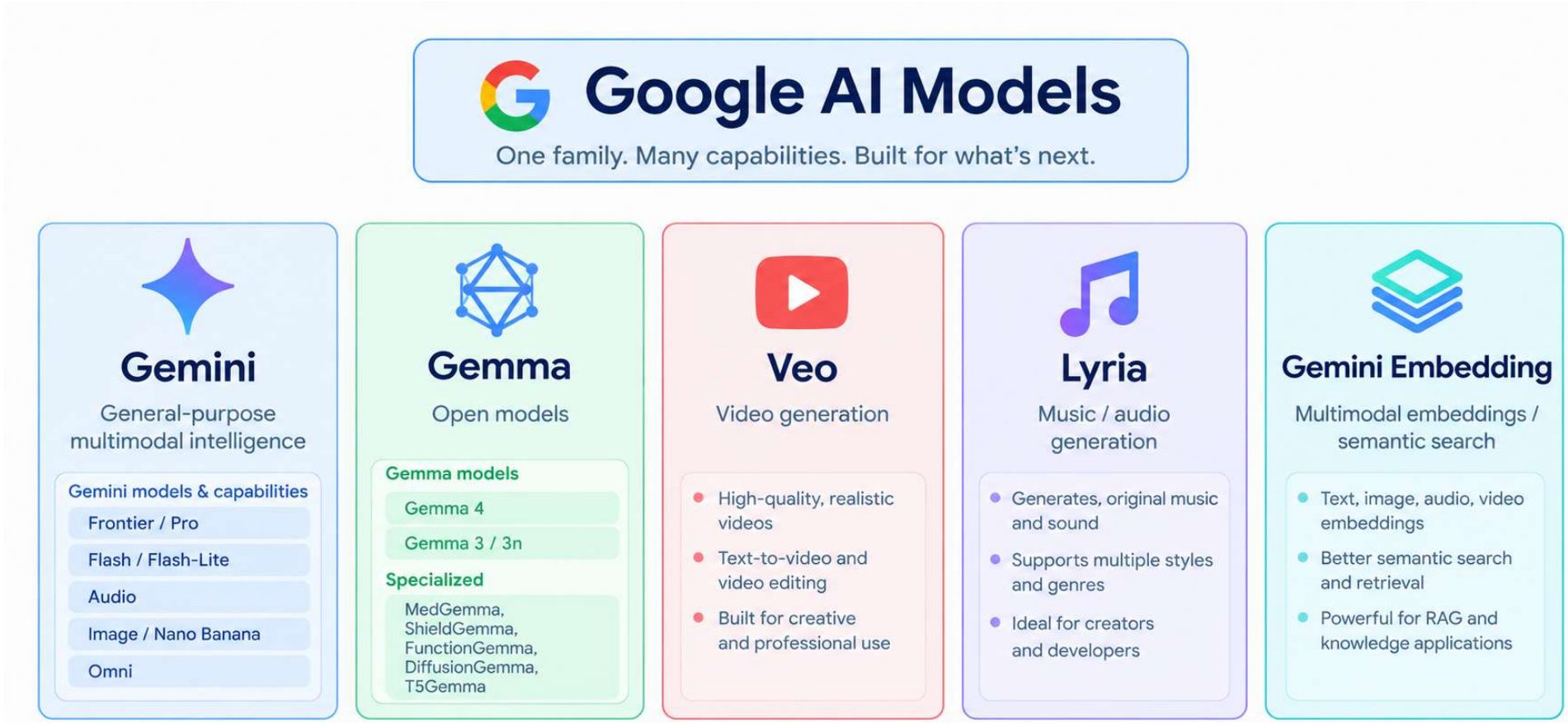
Evaluation framework

Agent Platform Agent Runtime, Cloud Run, GKE

Agent Identity, Model Armor, Agent Registry, Agent Gateway, Policies

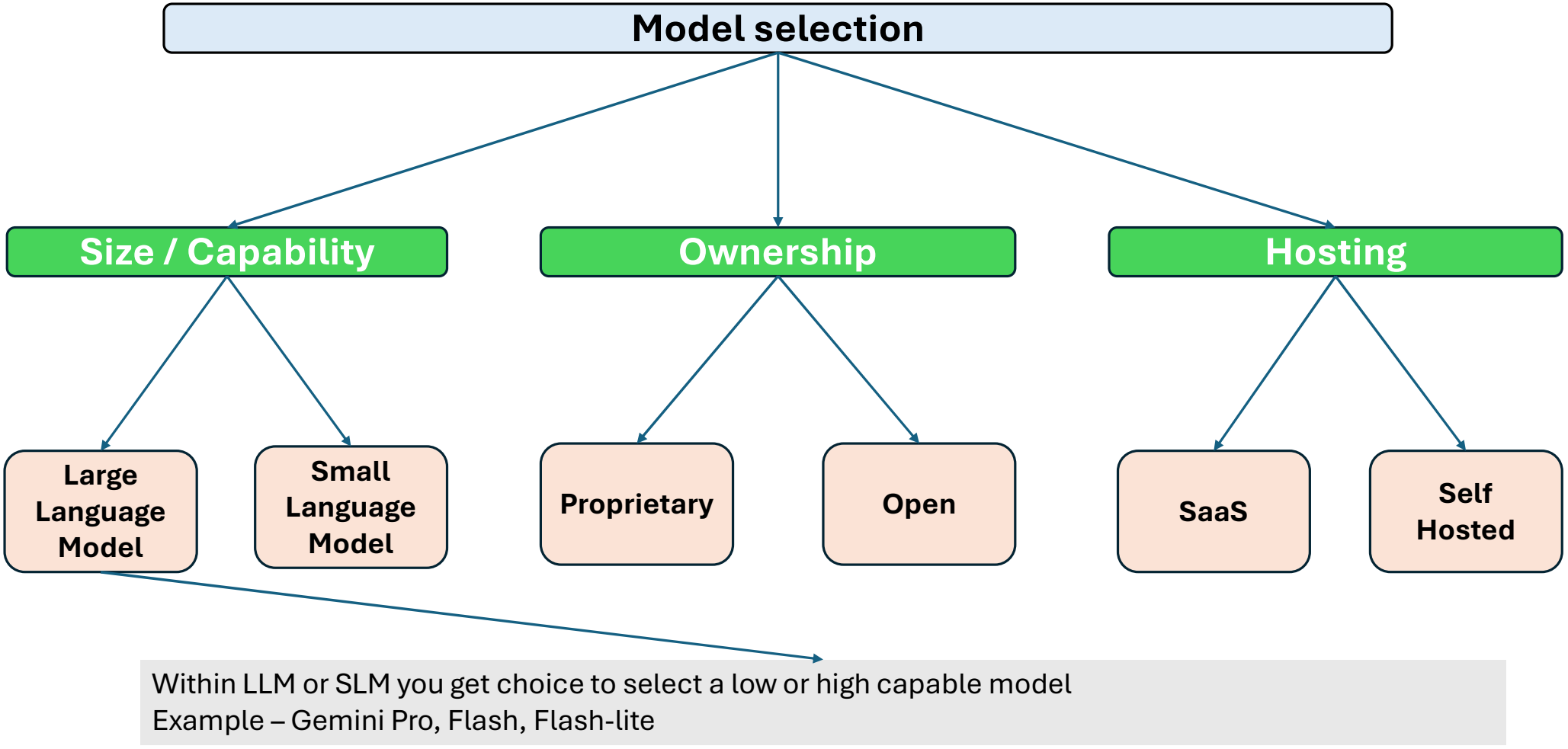
Cloud Logging, Monitoring, Trace

How do you choose which model to use?



How do you choose which model to use?

- **Cost** → tokens, infrastructure, traffic volume
- **Performance** → reasoning, tool calling, latency, accuracy
- **Security** → data sensitivity, residency, isolation
- **Customization** → fine-tuning, custom weights, serving requirements
- **Agent architecture** → single model, multiple models, specialized models
- **Operational complexity** → managed API vs infrastructure you operate



Within LLM or SLM you get choice to select a low or high capable model
Example – Gemini Pro, Flash, Flash-lite

Model by Size / Capability

	SLM – Small Language Model	LLM – Large Language Model
Parameters	Under 1 billon to few billons	100 of billions or trillions
Capability	Focused / simpler tasks	Complex reasoning & general tasks
Cost	Lower	Higher
Latency	Usually lower	Usually higher
Infrastructure	Lower requirements	Higher requirements
Good for agents	Simple, predictable tasks	Complex planning & tool use
Examples	Gemma	Gemini

Model by Ownership

	Open-Source / Open-Weight	Proprietary / Closed
Weights	Generally available	Not available
Customization	Usually more flexibility	Depends on provider
Control	Higher	Lower
Vendor dependency	Potentially lower	Higher
Responsibility	More on you	More on provider
Examples	Llama, Gemma, Qwen	Gemini, Claude

Model by Hosting

	SaaS / Managed API	Self-Hosted / Self-Deployed
Infrastructure	Provider manages it	You manage/control deployment
Setup	Easy	More complex
Operational effort	Low	High
Control	Lower	Higher
Security control	Provider's managed environment	Greater infrastructure/network control
Cost model	Pay for usage	Infrastructure + model serving
Best for	Variable workloads, rapid development	High volume, strict control, custom models

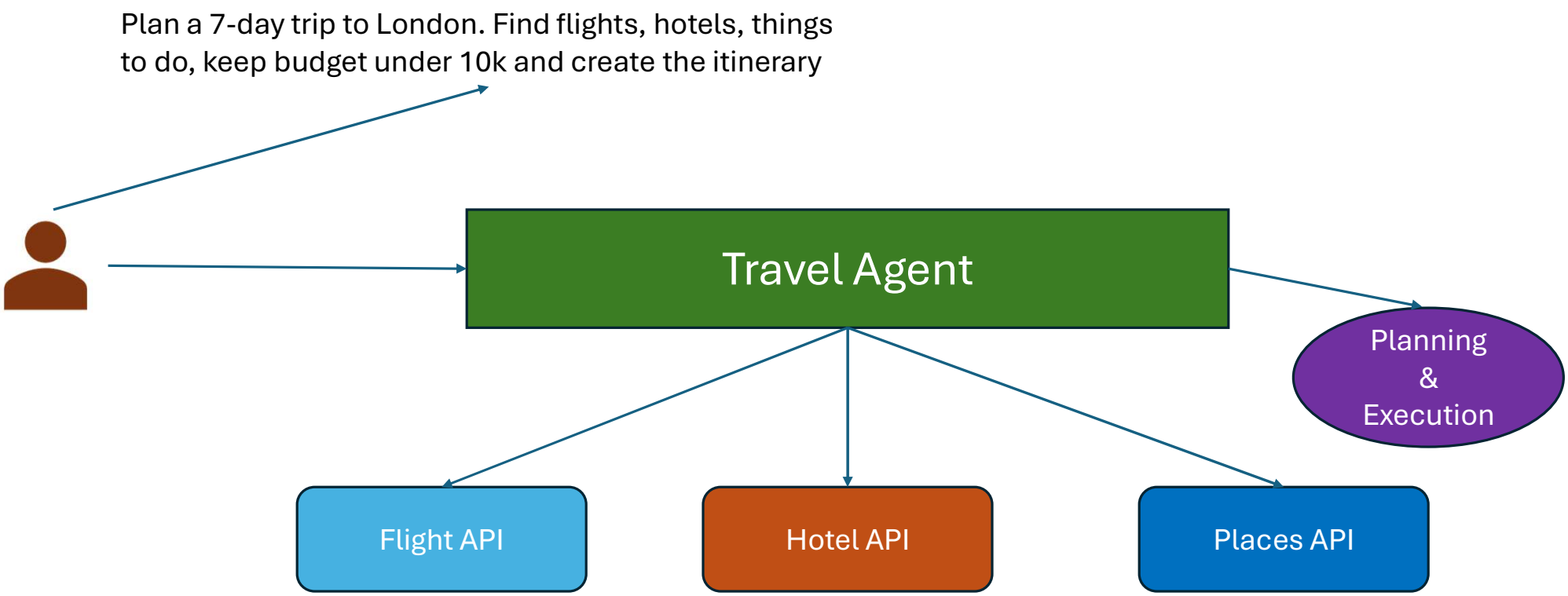
Question - A customer-service agent handles 50,000 requests per day. Most requests involve simple tasks such as checking order status, updating an address, and answering predefined policy questions. Only 5% require complex reasoning. Which type of model fit here optimizing cost and latency while maintaining the required capability?

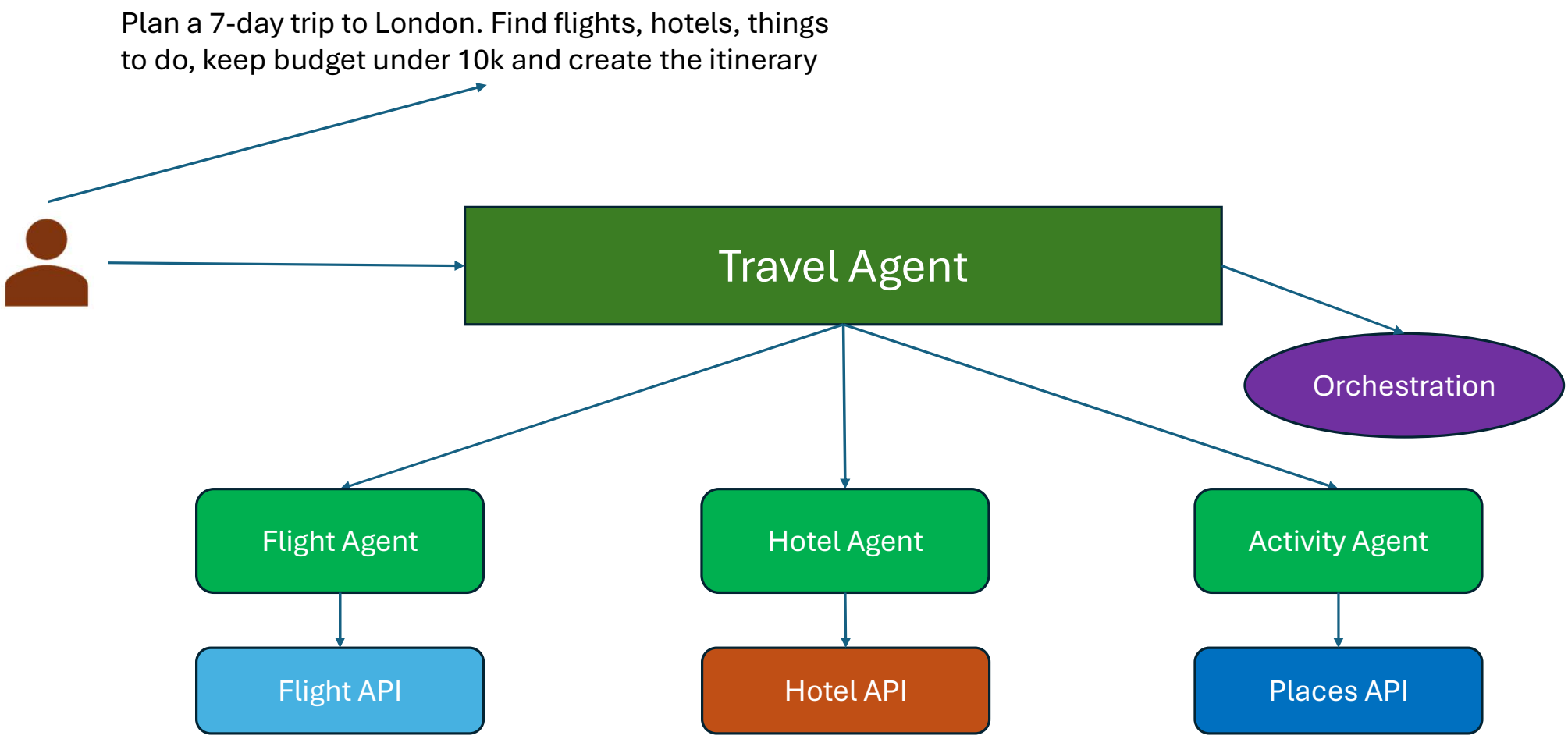
- **Large Language Model**
- **Small Language Model**

Question - An enterprise is building an AI agent that processes highly sensitive internal documents. Its security requirements prohibit sending the document contents to an external model provider.

- **SaaS proprietary LLM accessed through an API**
- **Self-hosted model deployed within the organization's controlled environment**

Multi-Agent System


Plan a 7-day trip to London. Find flights, hotels, things to do, keep budget under 10k and create the itinerary
Travel Agent
Planning & Execution
Flight API
Hotel API
Places API



The primary agent is called the **root agent** and others are called the **sub-agents**. These sub-agents can be local to the root agent or can be connected via a2a.

Gemini Enterprise
&
CX Agent Studio

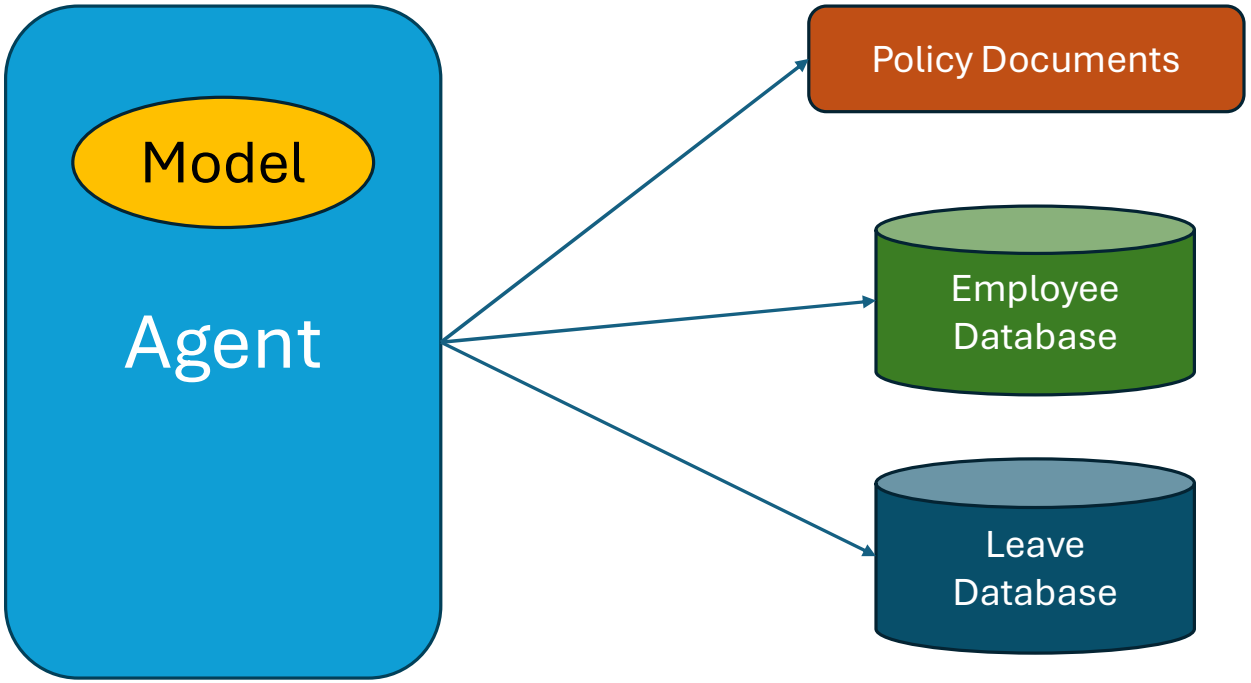
Suppose a company wants an **HR Employee Assistant**.

The agent should be able to answer:

Employee: "**How many vacation days do I have left and what is the HR policy on leave carry forward?**“

The agent needs to:

- Understand the question.
- Read the policy
- Look up the employee's information
- Call the HR system
- Retrieve the vacation balance
- Respond to the employee



What if you are using a 3rd party SaaS product for leave management, which has a custom authentication mode?

Custom Agent

No-Code / Low-Code Agent

Both are valid choices, you will have to decide based on timeline, available platforms etc

no-code / low-code agents Highlights:

Purpose	What it gives you
Faster development	Build an agent in hours/days rather than weeks
Less coding	Configure instructions, tools, knowledge, workflows, and integrations visually
Business-user friendly	Domain experts can create agents without being software developers
Rapid prototyping	Quickly test whether an agent can solve a business problem
Standardized architecture	Platform handles things like orchestration, sessions, authentication, deployment, etc.
Easy integration	Connect to enterprise data, APIs, SaaS applications, search, databases, etc.
Lower maintenance	Platform manages much of the underlying infrastructure
Governance	Enterprise platforms can provide built-in security, access control, monitoring and policies

Google’s product around no-code/low-code agent building:

- **Gemini Enterprise**
- **CX Agent Studio**
- **Dialogflow CX**
- **Dialogflow ES**

Imagine a large company with 20,000 employees. Information is scattered across

- HR policies
- IT documentation
- Google Drive
- Confluence
- ServiceNow/Jira
- Internal knowledge bases
- Finance/procurement documents

An employee asks:

"I am travelling to the US next month. What is the company's travel policy, what documents do I need, and how do I request a corporate credit card? And what about medical insurance"

Gemini Enterprise

The simplest definition:

Gemini Enterprise is Google's enterprise AI/agent platform for employees and organizations.
We are talking about internal data of an organization

In a better way:

Google Cloud’s comprehensive enterprise AI and agentic platform designed to bring advanced generative AI, intelligent search, and autonomous workflows to organizations. It provides a single **Front Door** where employees can search company information, use AI assistants, and discover/run AI agents, while organizations can connect enterprise data, govern access, and automate workflows

Important to remember – This caters to the internally accessible data

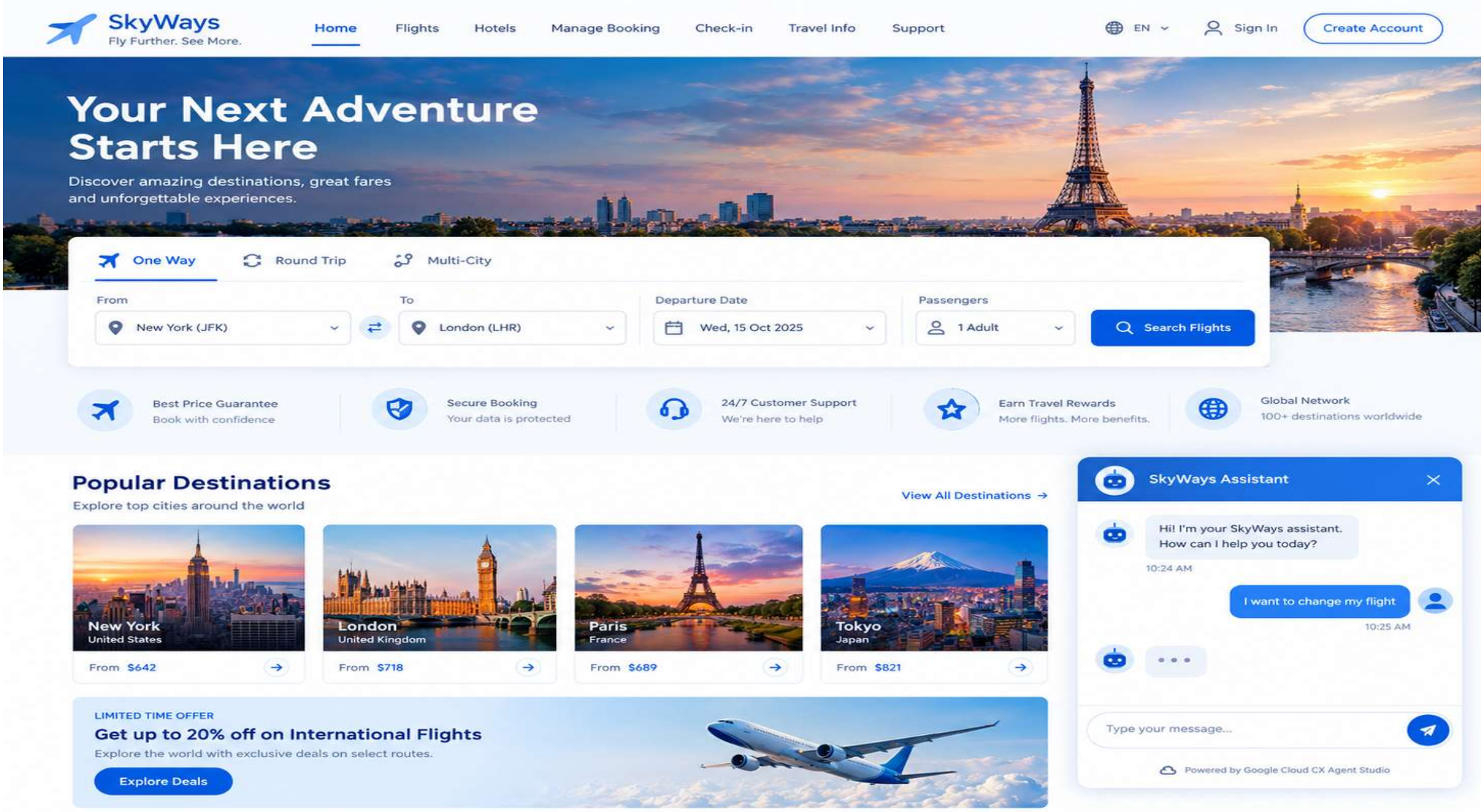
Gemini Enterprise

Core Capabilities:

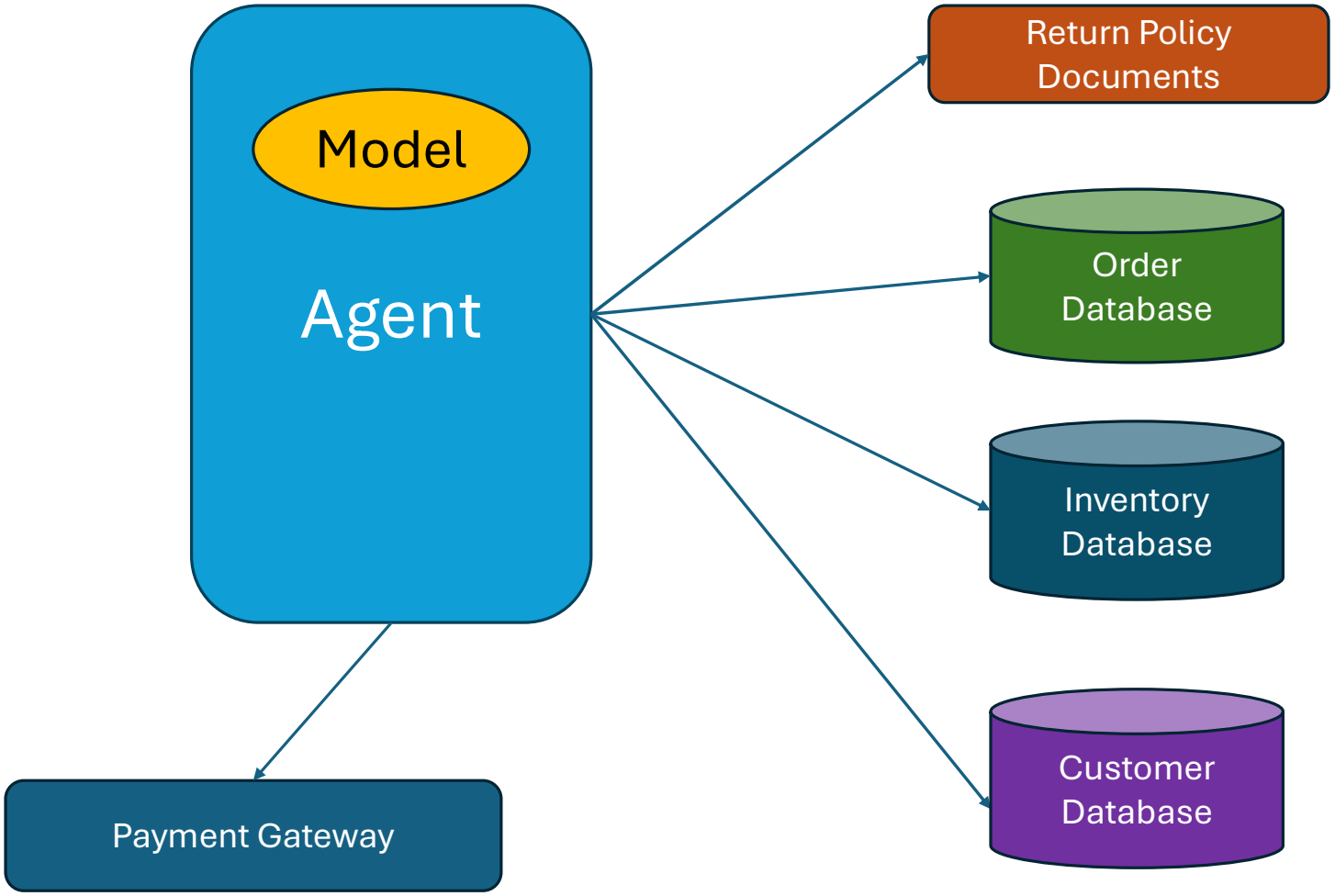
- **Intelligent Enterprise Search -** Provides a permissions-aware, multimodal search engine that allows workers to query internal documentation, data stores, and connected repositories to find precise answers and documents instantly

- **Conversational Assistant -** Powered by Google's advanced Gemini models, it assists employees with daily tasks such as drafting reports, analysing complex spreadsheets, generating content (text, images, and videos), and summarizing long meetings or documents.

- **Agentic Workflow Automation -** Features a no-code/low-code workflow builder that lets organizations create, customize, and deploy autonomous AI agents. These agents can execute multi-step tasks and coordinate actions across different business applications


SkyWays
Fly Further. See More.
Home
Flights
Hotels
Manage Booking
Check-in
Travel Info
Support
EN
Sign In
Create Account
Your Next Adventure Starts Here
Discover amazing destinations, great fares and unforgettable experiences.
One Way
Round Trip
Multi-City
From
New York (JFK)
To
London (LHR)
Departure Date
Wed, 15 Oct 2025
Passengers
1 Adult
Search Flights
Best Price Guarantee
Book with confidence
Secure Booking
Your data is protected
24/7 Customer Support
We're here to help
Earn Travel Rewards
More flights. More benefits.
Global Network
100+ destinations worldwide
Popular Destinations
Explore top cities around the world
View All Destinations →
New York
United States
From $642
London
United Kingdom
From $718
Paris
France
From $689
Tokyo
Japan
From $821
LIMITED TIME OFFER
Get up to 20% off on International Flights
Explore the world with exclusive deals on select routes.
Explore Deals
SkyWays Assistant
Hi! I'm your SkyWays assistant. How can I help you today?
10:24 AM
I want to change my flight
10:25 AM
Type your message...
Powered by Google Cloud CX Agent Studio


DUMMYTOYS - Your Haven for Playtime!
EXPLORE
FUN & GAMES!
New Arrivals & Summer Deals.
DUMMYTOYS
Categories
Sales
New Arrivals
Gift Cards
Search toys...
Age
6 - 12 years
Type
Action Figures
Dolls
Games
Brand
Rainbow Building Blocks
$24.99
(126)
Add to Cart
Cuddly Teddy Bear
$19.99
(128)
Add to Cart
Galaxy Explorer Rocket
$34.99
(128)
Add to Cart
Dino Discovery Set
$49.99
(129)
Add to Cart
ToyBot Chat
How can I help you?
I have received a broken toy, how do I replace / refund?
11:12 AM
Oh no, I'm so sorry to hear that! We can certainly help. Do you have your order number, and could you please share a photo of the damage?
11:15 AM
Type your question...
MacBook Pro



If return has to be issued

Don't forget we are talking about a full-stack agent here. You will have to spend time and effort for UI development and Agent and UI integration as well

Conversational Agents

CX Agent Studio

This is a specialized, **low-code** development environment specifically dedicated to building customer-facing conversational agents (chatbots and voicebots)

- Shorten time to value. Rapid development
- Lower operating costs
- Multi-modal engagement
- Omnichannel presence
- Default connectors + custom code
- Development + Evaluation + Deployment + Monitor

Drag-and-Drop visual builder

Custom Agents

When to use a custom / high-code agent:

- You need complex, multi-step reasoning
- You need to integrate custom tools or APIs
- You need control over the agent's behaviour
- You need multiple specialized agents with a sophisticated orchestration
- The task is dynamic rather than deterministic
- You want to use advanced feature like session, memory, callback
- You want more flexibility with Model selection

Few popular Agent Framework for custom agents:

- Google ADK (Agent Development Kit)
- LangGraph
- Microsoft Agent Framework
- CrewAI
- OpenAI Agents SDK
- LlamaIndex

Why Google ADK

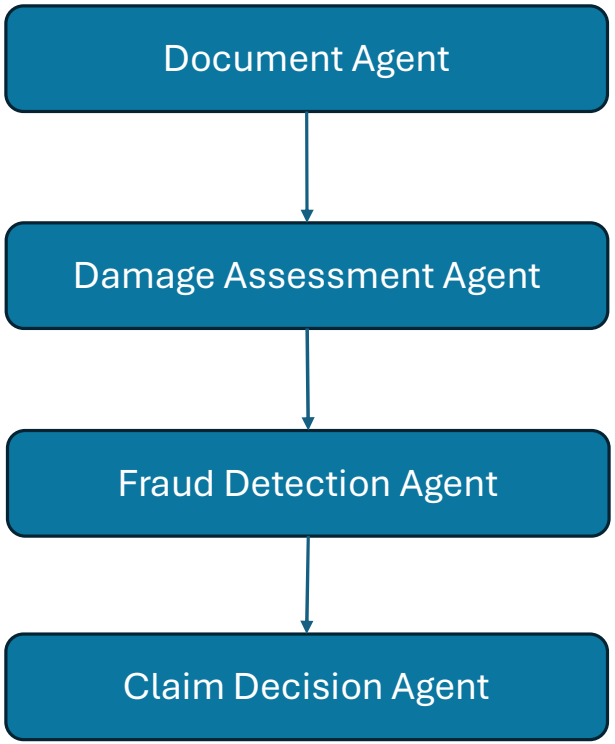
- Google Cloud–native
- Open Source, Code-first and highly customizable
- Built for multi-agent systems - advanced orchestration workflow agents
- Model flexibility (Beyond Gemini)
- Strong tool integration
- Built-in support for agent-to-agent (a2a) communication
- MCP support
- Evaluation and observability
- Unlocks advanced features in Gemini Enterprise Agent Platform

Multi-Agent Workflows:

- Sequential Workflow
- Parallel Workflow
- Loop Workflow
- Dynamic

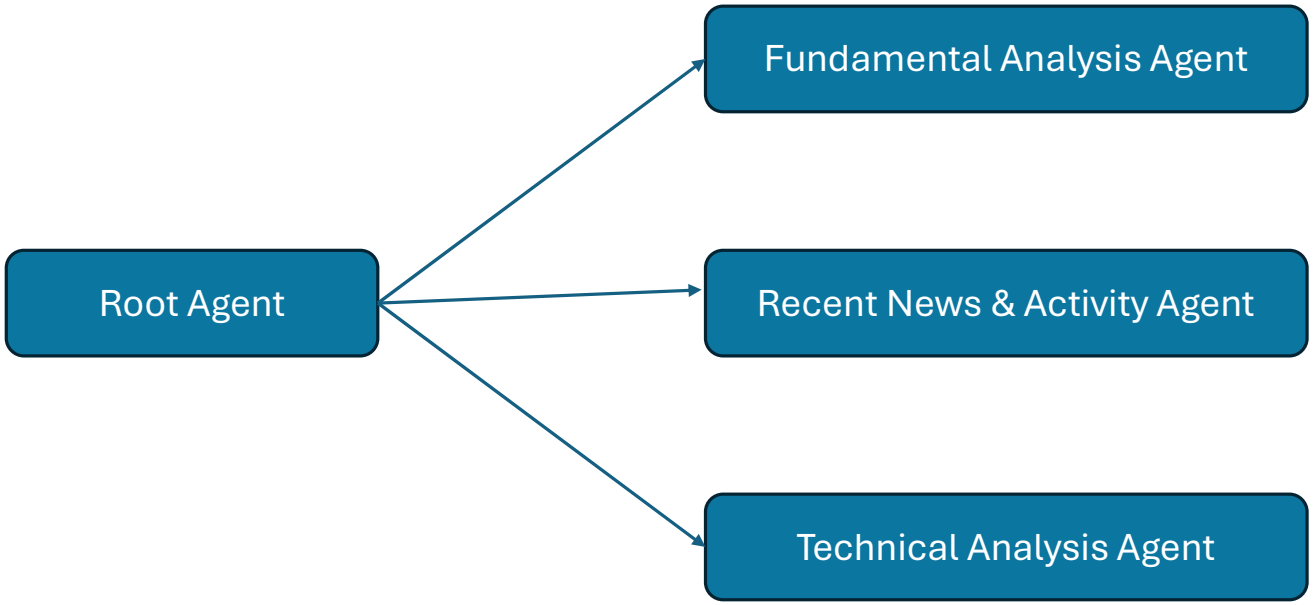
Sequential Workflow

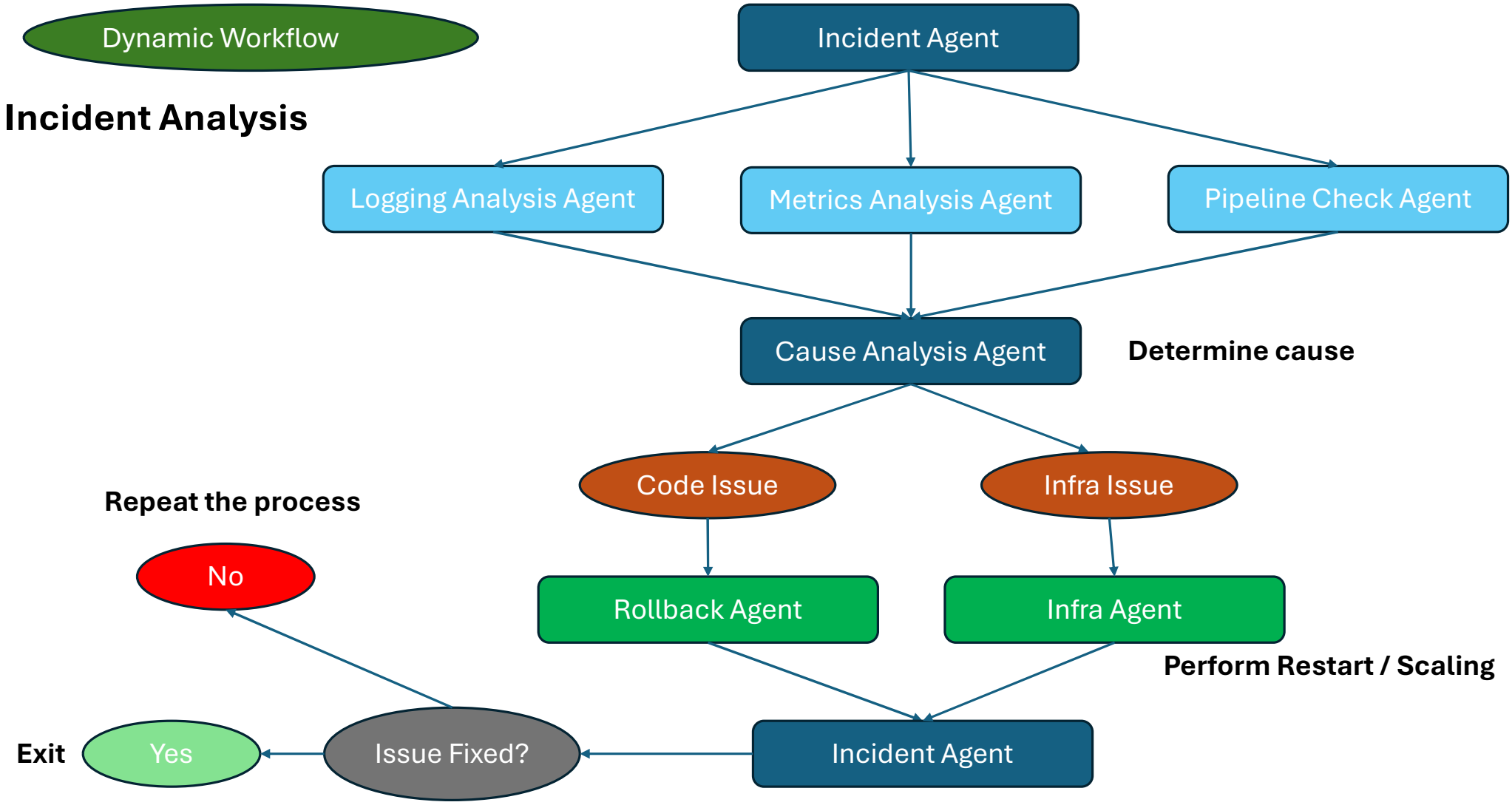
Insurance claim processing



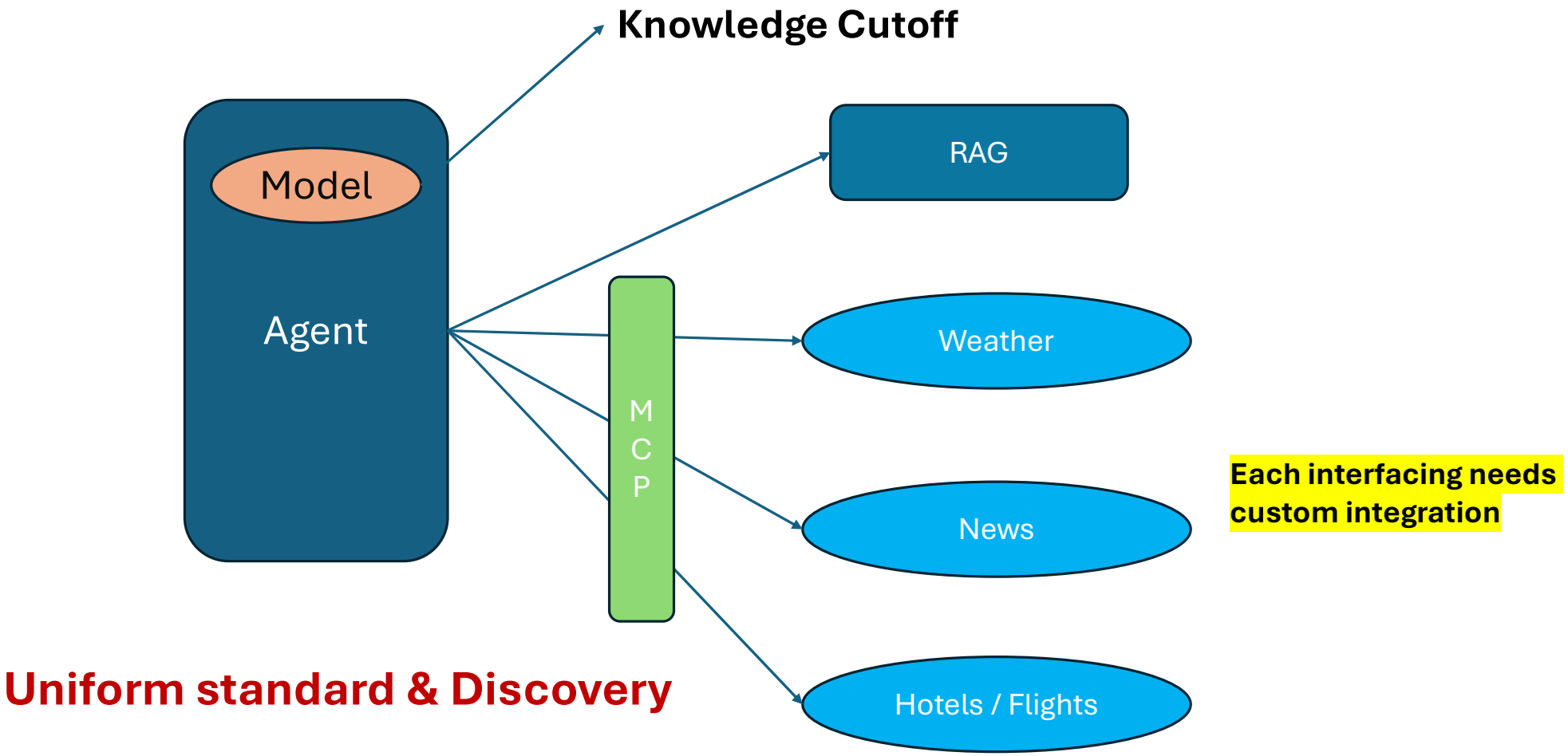
Parallel Workflow

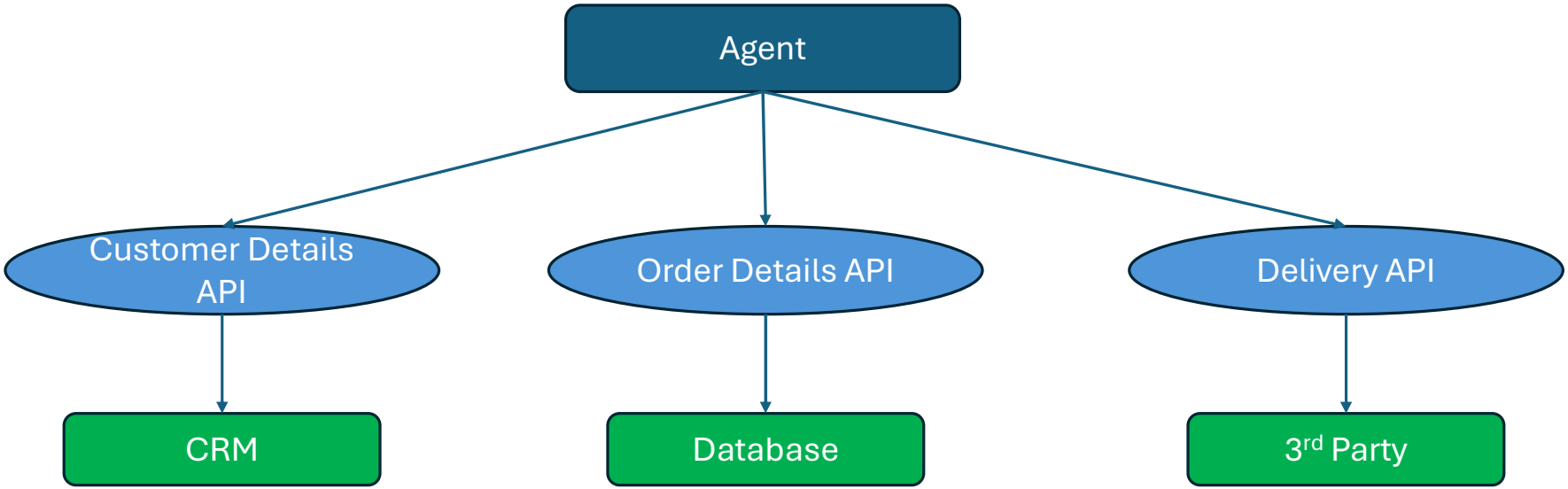
Investment Decision



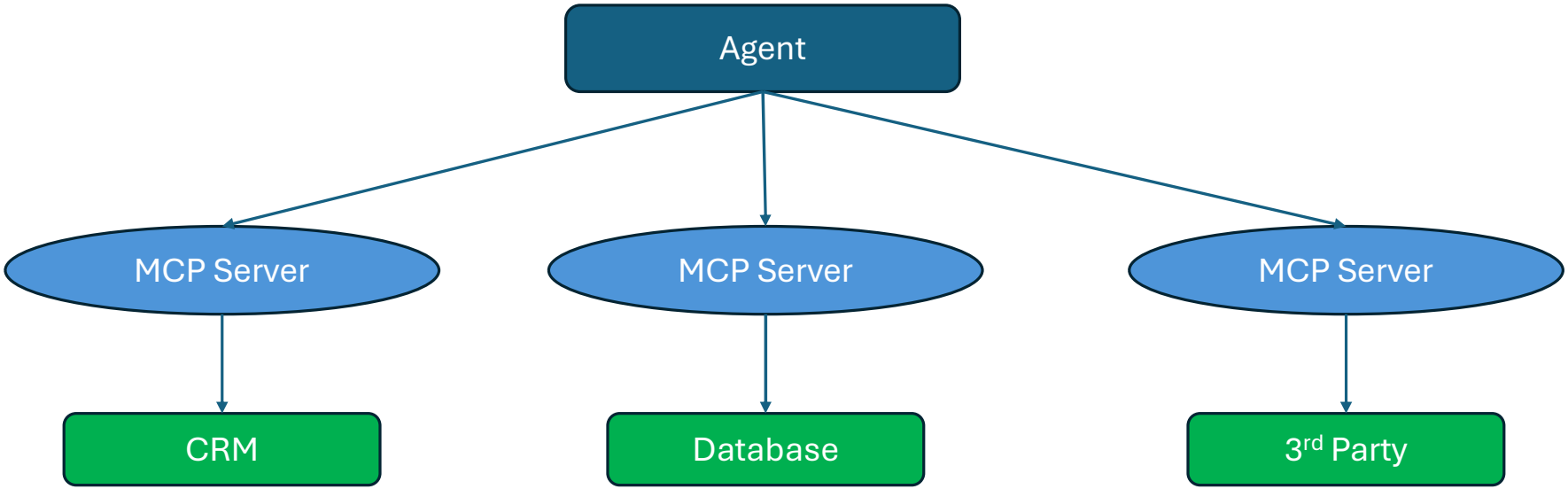

Dynamic Workflow
Incident Analysis
Incident Agent
Logging Analysis Agent
Metrics Analysis Agent
Pipeline Check Agent
Cause Analysis Agent
Determine cause
Code Issue
Infra Issue
Repeat the process
No
Rollback Agent
Infra Agent
Perform Restart / Scaling
Exit
Yes
Issue Fixed?
Incident Agent

MCP


Knowledge Cutoff
Model
Agent
RAG
M
C
P
Weather
News
Hotels / Flights
Each interfacing needs custom integration
Uniform standard & Discovery



Problem with this approach is that you will have to write custom integration with each of these systems and map the right request with the right method

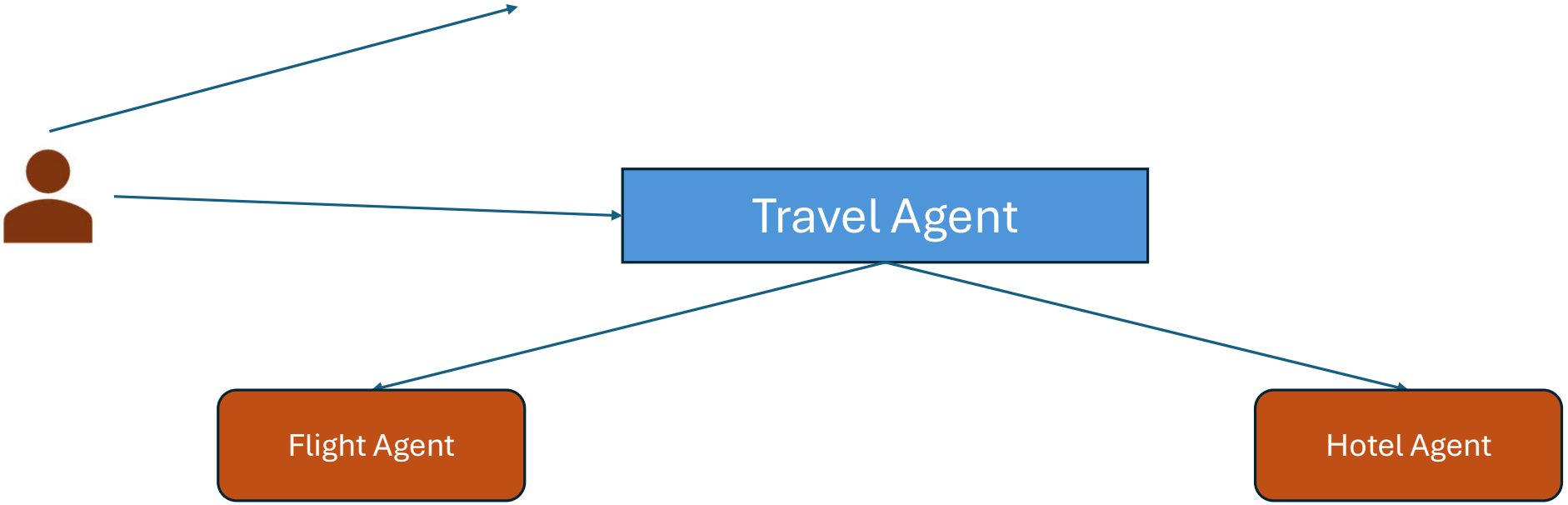


The **MCP server exposes capabilities in a standardized way**, so the agent doesn't need a bespoke integration for every system.

MCP standardizes how AI applications discover and interact with external tools, data, and capabilities.

A2A

Plan a 7-day trip to London. Find flights, hotels, things to do, keep budget under 10k and create the itinerary



- **Flight Agent & Hotel Agent deployed as an independent agent as a separate deployment**

How do the agents communicate? Normal HTTP API calls?



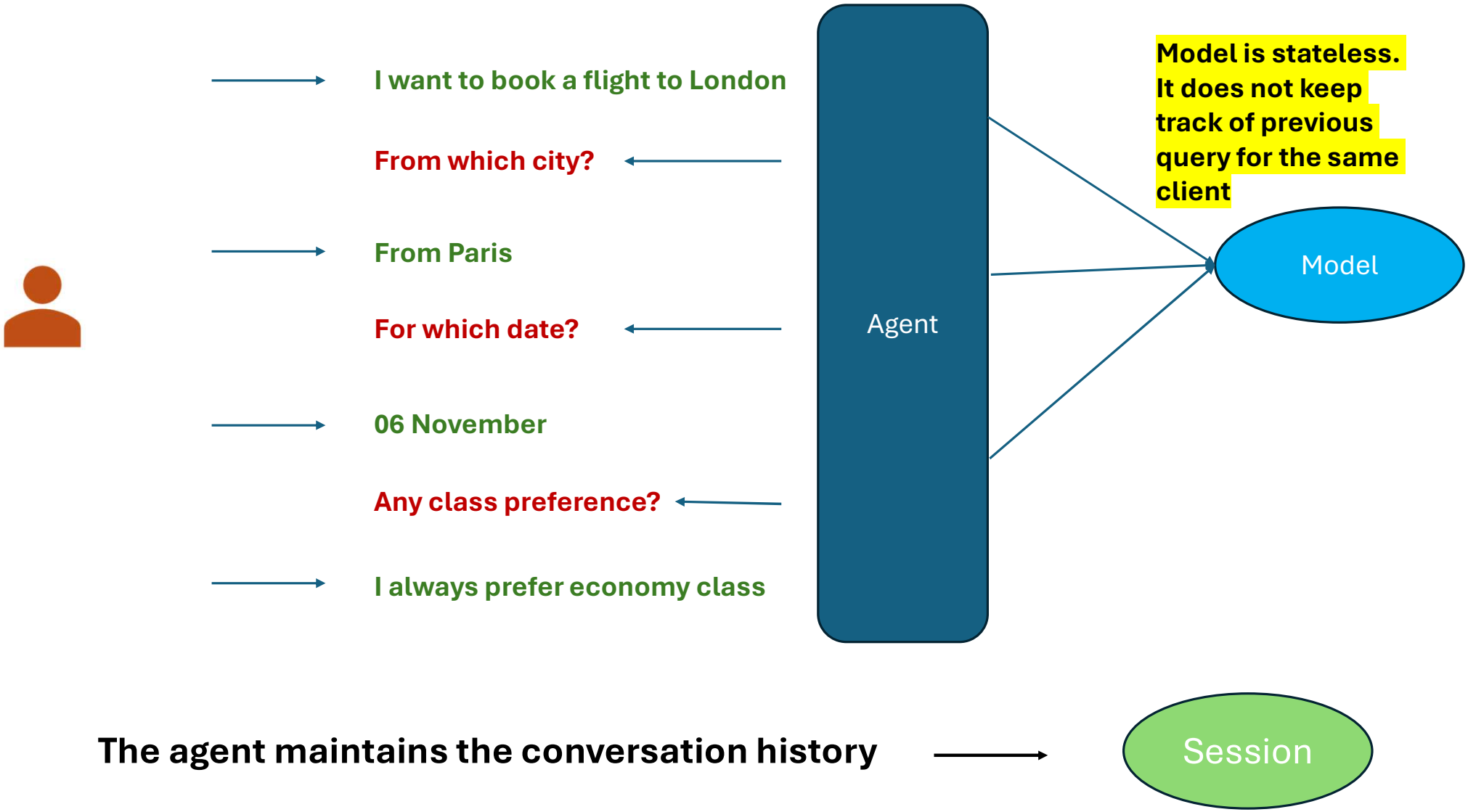
Benefit of A2A:

- **Compatibility** - Cross-vendor, cross-framework interoperability
- **Agent Discovery** - Agents can discover what other agents are capable of doing – concept of agent card
- **Agent Modularity** - Focused agents perform specific tasks
- **Long-running task handling** - asynchronous messaging and status tracking, so agents can spin off tasks and check back later without timing out.

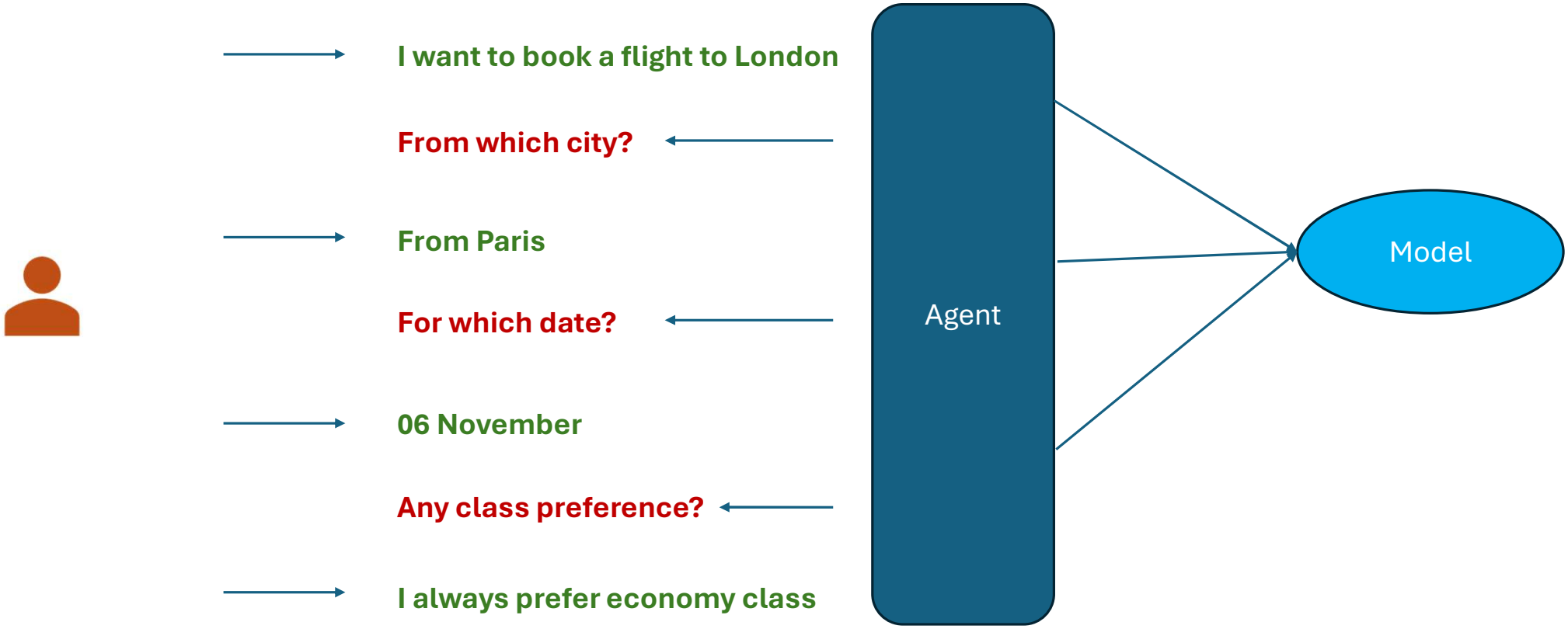
MCP vs A2A

- MCP connects an **agent to tools and data** - More like Integration
- A2A connects **agents to other agents** - More like Collaboration
- A typical architecture uses both: agents talk to each other via A2A, and each uses MCP internally to reach its tools

Session
&
State
&
Memory



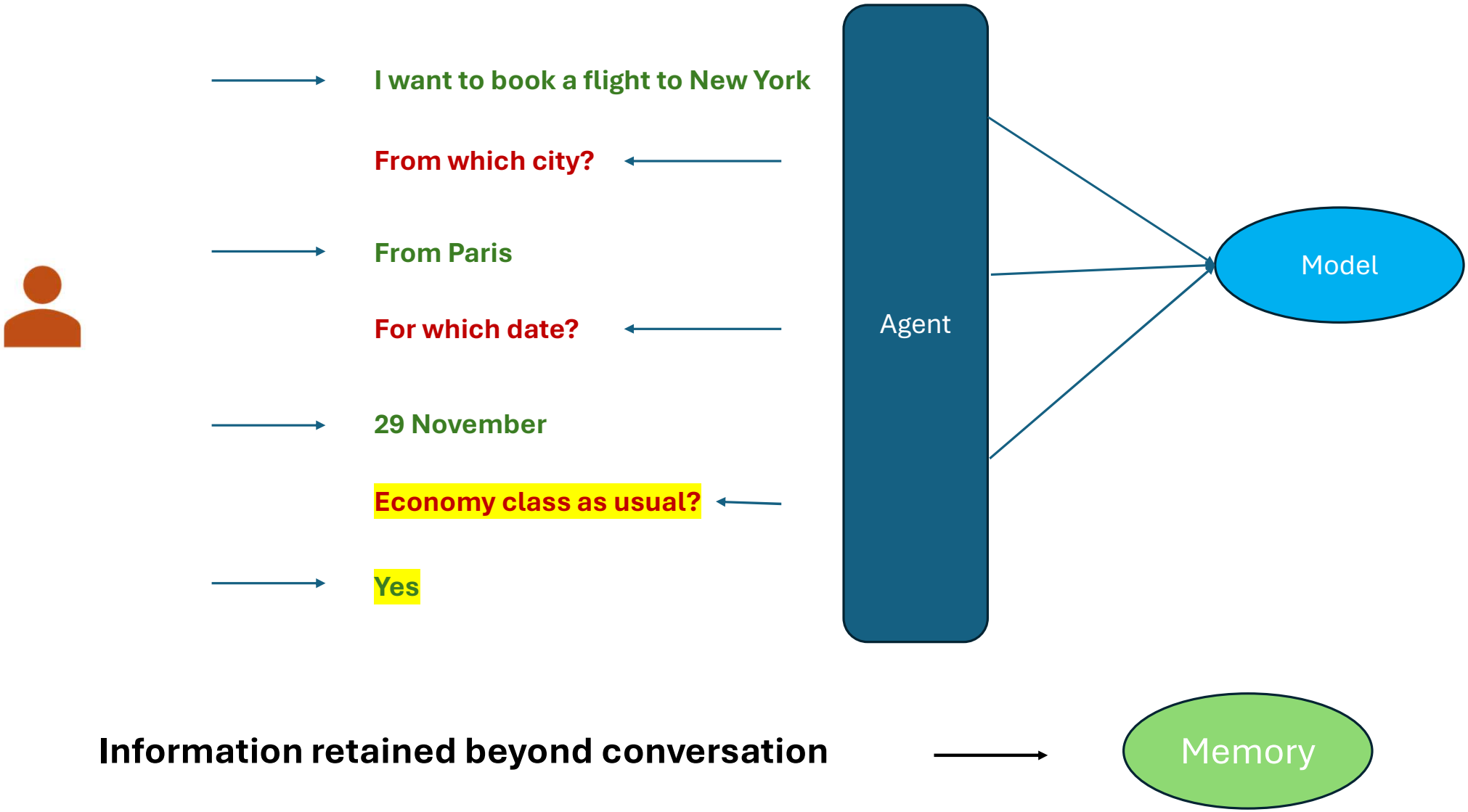
The agent maintains the conversation history → Session

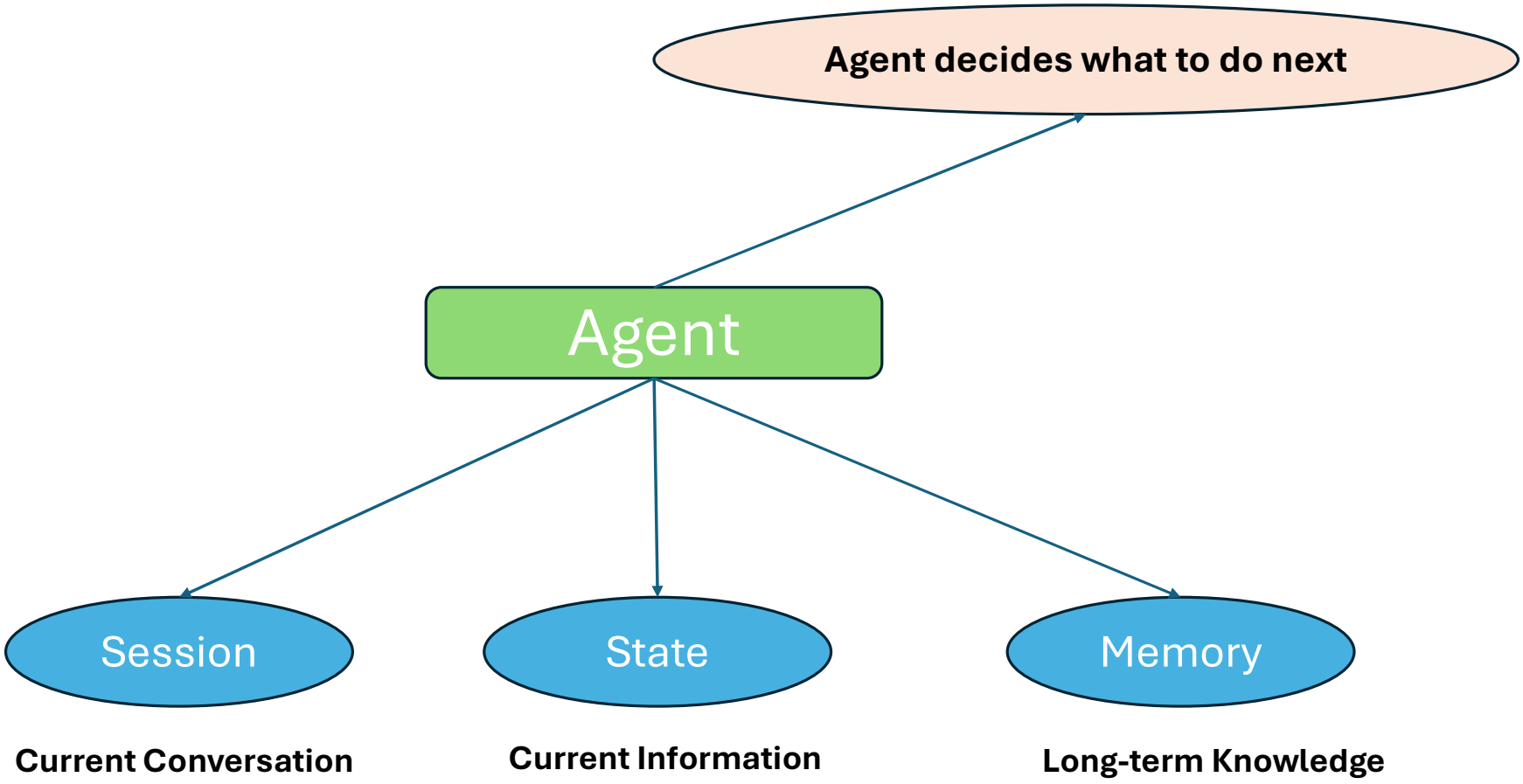


Destination – London
Source – Paris
Date – 6th November
Class – Economy

The agent keeps track of the information shared




I want to book a flight to New York
From which city?
From Paris
For which date?
29 November
Economy class as usual?
Yes
Agent
Model
Information retained beyond conversation
Memory


Agent decides what to do next
Agent
Session
State
Memory
Current Conversation
Current Information
Long-term Knowledge

Concept	Think of it as	Agent's Perspective	Lifetime
Session	The conversation	What conversation am I in?	Current interaction
State	Working information	What do I know about the current task?	Usually current session
Memory	Long-term knowledge	What do I know from previous interactions?	Across sessions

Informally:

Session – Short-term Memory

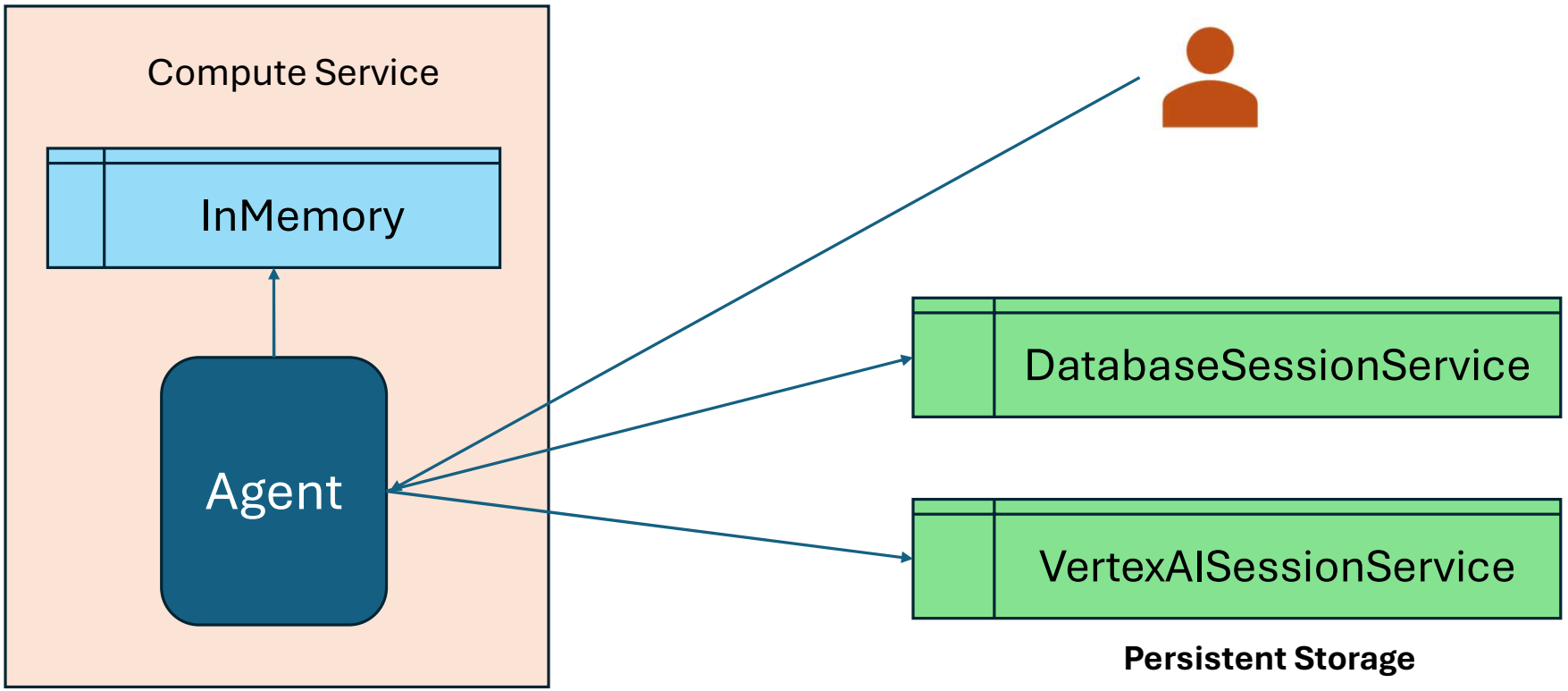
Memory – Long-term Memory

How GCP manages session for Agents?

SessionService

- InMemorySessionService
- VertexAiSessionService
- DatabaseSessionService

SessionService



Features	InMemorySessionService	VertexAiSessionService	DatabaseSessionService
Where data lives	In your application's memory	Google Cloud / Agent Platform managed Sessions	Google Cloud / Database
Persistence	Lost when application restarts	Persistent	Persistent
Usage	Mostly development/testing	Production	Production
Scaling	Limited	Designed for scalable deployments	Designed for scalable deployments
Managed by Google Cloud	No	Yes	Partly
Typical use	Testing, demos, prototyping	Production agents	Production agents
Survives restart/deployment	**No**	**Yes**	**Yes**

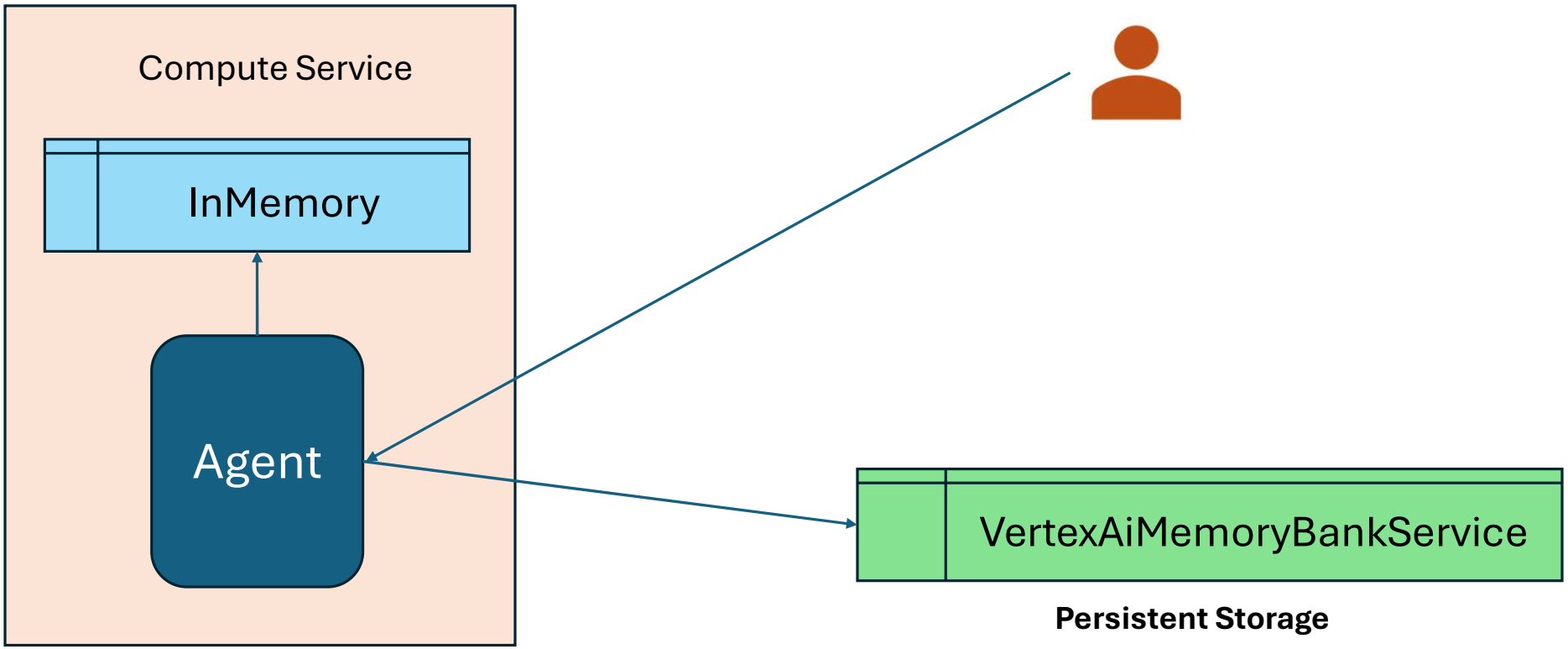
How GCP manages Memory for Agents?

MemoryService & MemoryBankService

- InMemoryMemoryService
- VertexAiMemoryBankService

You can custom develop your memory service. Choose any suitable storage service (DB, Buckets) and persist the memory

MemoryService



Feature	InMemoryMemoryService	VertexAiMemoryBankService
Purpose	Simple memory for development and testing	Long-term memory for production agents
Storage location	Application RAM	Google Cloud Memory Bank
Persistence	Lost when the application restarts	Persists across sessions and restarts
Memory creation	Stores memory entries provided by the application	Can extract and manage useful information from conversations
Memory retrieval	Retrieves available in-memory entries	Retrieves relevant memories using semantic search
Multiple instances	Memory isn't automatically shared between instances	Supports shared, cloud-managed memory
GCP configuration	Not required	Requires Google Cloud configuration and permissions
Best suited for	Local development, demos, prototypes and testing	Production agents requiring personalized, long-term context

Agent Deployment

Choices for Deployment

- Agent Runtime (Preferred Choice)
- Cloud Run
- GKE Cluster

Agent Runtime

Fully managed infrastructure. Instant scale-to-zero with zero server provisioning

Cloud Run

Containerized flexibility. Serverless execution for custom agent frameworks.

GKE Cluster

Enterprise orchestration. Maximum control for multi-agent distributed clusters.

Agent Runtime

Capabilities

- **Native State Management:** Built-in memory and sessions
- **Zero Infrastructure:** No clusters, Docker files, or scaling policies to configure
- **Fast Iteration:** Immediate deployment from workflow definitions
- **Agent Platform Security:** Enterprise security handled by the platform
- **Integrated Evaluation:** Online evaluation pipelines for predefined & custom attributes
- **Supports BYOA:** Deploy container images from other framework

Trade-offs

- **Control:** Less control over infrastructure and networking
- **Only for Agents:** Tuned for agent workloads, not general services
- **Customization:** Platform-defined configuration and limits
- **Build for ADK:** Get full benefit if agents are built with ADK framework

Cloud Run

Capabilities

- **Container Flexibility:** Pack any custom libraries, frameworks, or dependencies
- **Pay-per-Use:** Scale to zero when idle with automatic concurrency scaling
- **Secure Integration:** Native VPC connectivity and IAM security controls
- **Common Platform:** Shared architecture with order services (ex: MCP Server)

Trade-offs

- **Stateless**: externalize sessions, memory and secrets
- **Configuration:** You assemble auth and observability yourself
- **Timeout:** Request timeouts limit for very long-running tasks

GKE Cluster

Capabilities

- **Granular Control:** Fine-tuned resource usage, GPU affinity, and scheduling
- **Multi-Agent Meshes:** Scalable communication across distributed agent swarms
- **Persistent State:** High-performance volumes for vector databases and caching
- **Infrastructure:** Full control of nodes, networking and security

Trade-offs

- **Operational overhead:** highest with clusters setup, maintenance and upgrades
- **Slowest path:** Terraform, image build, kubectl
- **Stateless:** externalize sessions and memory
- **Expensive**: Cluster maintenance cost + operational cost

Criteria	Agent Runtime	Cloud Run	Google Kubernetes Engine (GKE)
Primary purpose	Managed runtime for AI agents	Serverless container hosting	Kubernetes-based container orchestration
Best suited for	Agents needing managed runtime capabilities	Containerized agents and APIs with minimal infrastructure management	Complex distributed agent systems needing infrastructure control
Infrastructure management	Google manages the runtime	Google manages serverless infrastructure	You manage Kubernetes workloads and configuration
Deployment approach	Deploy supported agent code using Agent Engine APIs or SDKs	Deploy source code or container images	Deploy containers using Kubernetes manifests, Helm or CI/CD
Agent frameworks	Best suited for ADK. Support any framework	ADK, LangGraph and other container-compatible frameworks	ADK, LangGraph and other container-compatible frameworks
Scaling	Managed agent-runtime scaling	Automatic scaling, including scale-to-zero for suitable services	HPA, VPA and cluster autoscaling
Operational effort	Low	Low to moderate	High

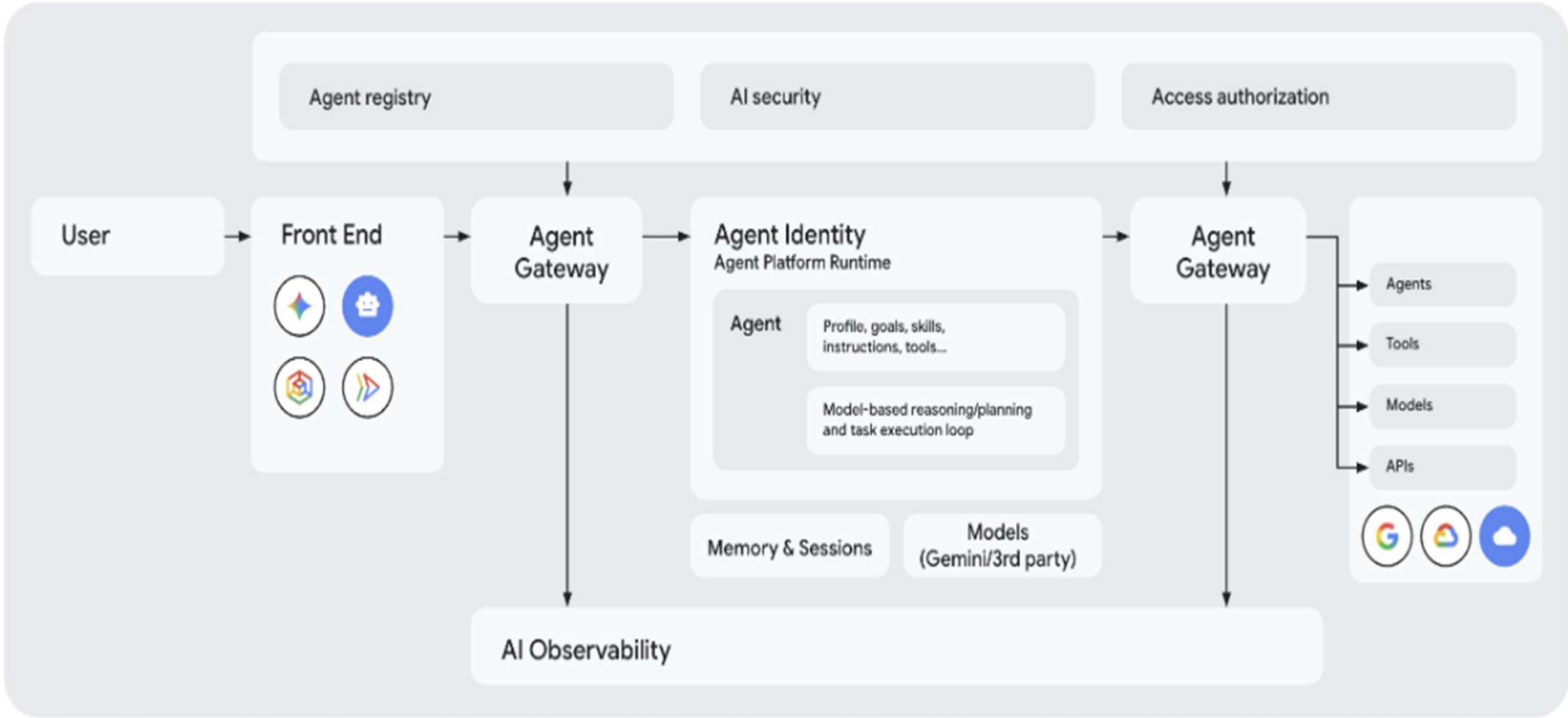
Criteria	Agent Runtime	Cloud Run	Google Kubernetes Engine (GKE)
Sessions, State & Memory	Managed session integration is available	Configure external session storage or integrate managed services	Configure external session storage or integrate managed services
Long-term memory	Can integrate managed memory capabilities	Integrate a memory service as needed	Integrate a memory service as needed
Custom runtime requirements	More constrained runtime environment	Flexible container configuration	Highest level of infrastructure and workload configuration
Typical use case	ADK agent using managed sessions and memory	Containerized chatbot, API or MCP server	Enterprise agent platform with specialized orchestration
Long-term memory	Can integrate managed memory capabilities	Integrate a memory service as needed	Integrate a memory service as needed
Best exam clue	Managed agent hosting	Serverless containers and scale-to-zero	Kubernetes control and custom orchestration

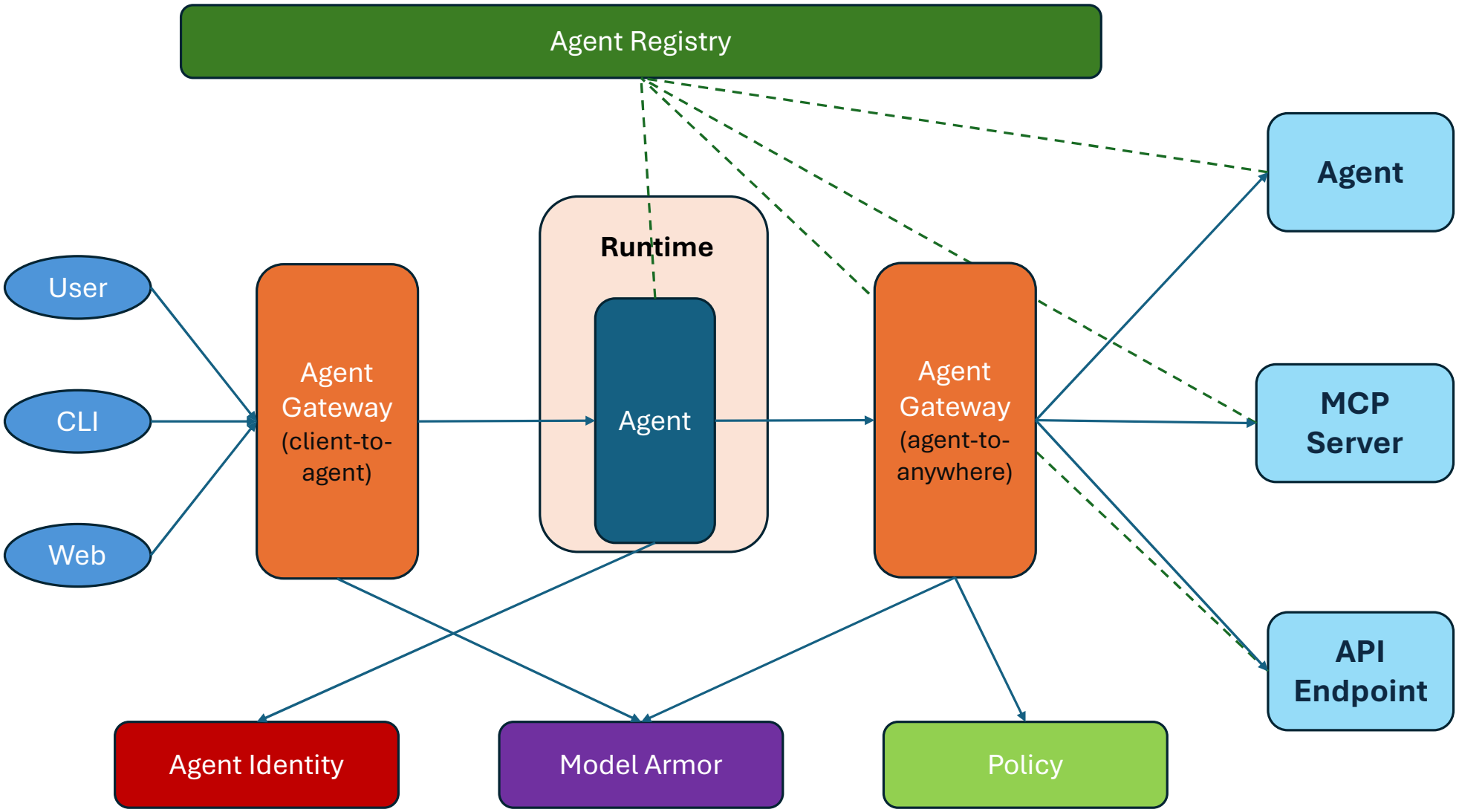
Agent Governance & Security

Agent Governance & Security:

- Agent Registry
- Agent Identity
- Model Armor
- Agent Gateway
- Policies

Component	What it is for	The Question
Agent Registry	A catalogue of agents, MCP servers, skills and endpoints.	Where agents are discovered
Agent Identity	A unique SPIFFE-based ID for every agent, in place of shared service accounts.	How agents are identified
Model Armor	Screens prompts, tool payloads and responses for injection and data leaks.	How AI interactions are screened/protected
Agent Gateway	The enforcement point for traffic into and out of agents.	How agents are accessed and controlled
Policies	Decides what is allowed and what is not from an agent	What the agent is allowed to do


Agent registry
AI security
Access authorization
User
Front End
Agent Gateway
Agent Identity
Agent Platform Runtime
Agent
Profile, goals, skills, instructions, tools...
Model-based reasoning/planning and task execution loop
Memory & Sessions
Models (Gemini/3rd party)
Agent Gateway
Agents
Tools
Models
APIs
AI Observability


Agent Registry
Agent
Runtime
User
CLI
Web
Agent Gateway (client-to-agent)
Agent
Agent Gateway (agent-to-anywhere)
MCP Server
API Endpoint
Agent Identity
Model Armor
Policy

Evaluation

You want to test a **Traditional Software**

- Unit / System Testing

→ The steps are pre-defined and controlled

AI Agents path is **non-deterministic**

- Agent decides **on the go** what to use and how to process a request using tools, sub-agents
- The answer will be different every time with respect to to the content

Traditional testing process will not be effective for AI Agents

Evaluation

What do we evaluate?

- **Final Response** (**End output of the agent**)

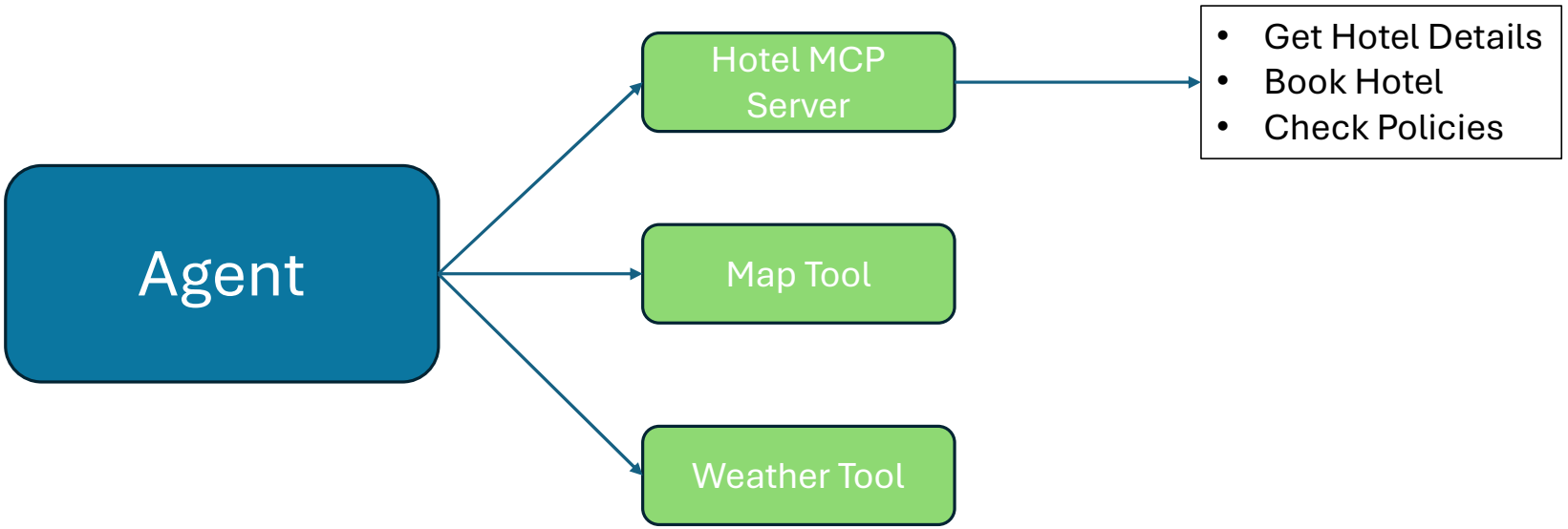
  Was the goal achieved, accurately, safely and in the right tone?

- **Trajectory** (**Path of execution**)

  Did it call the right tools, in a sensible order(no necessarily always the same order), with the right inputs?

Let’s understand with an example agent > **Hotel Agent**

System Instructions: You are an agent who helps with hotel booking. User might provide budget and weather constraint for a booking. Use the available tools for weather, map and hotel related queries

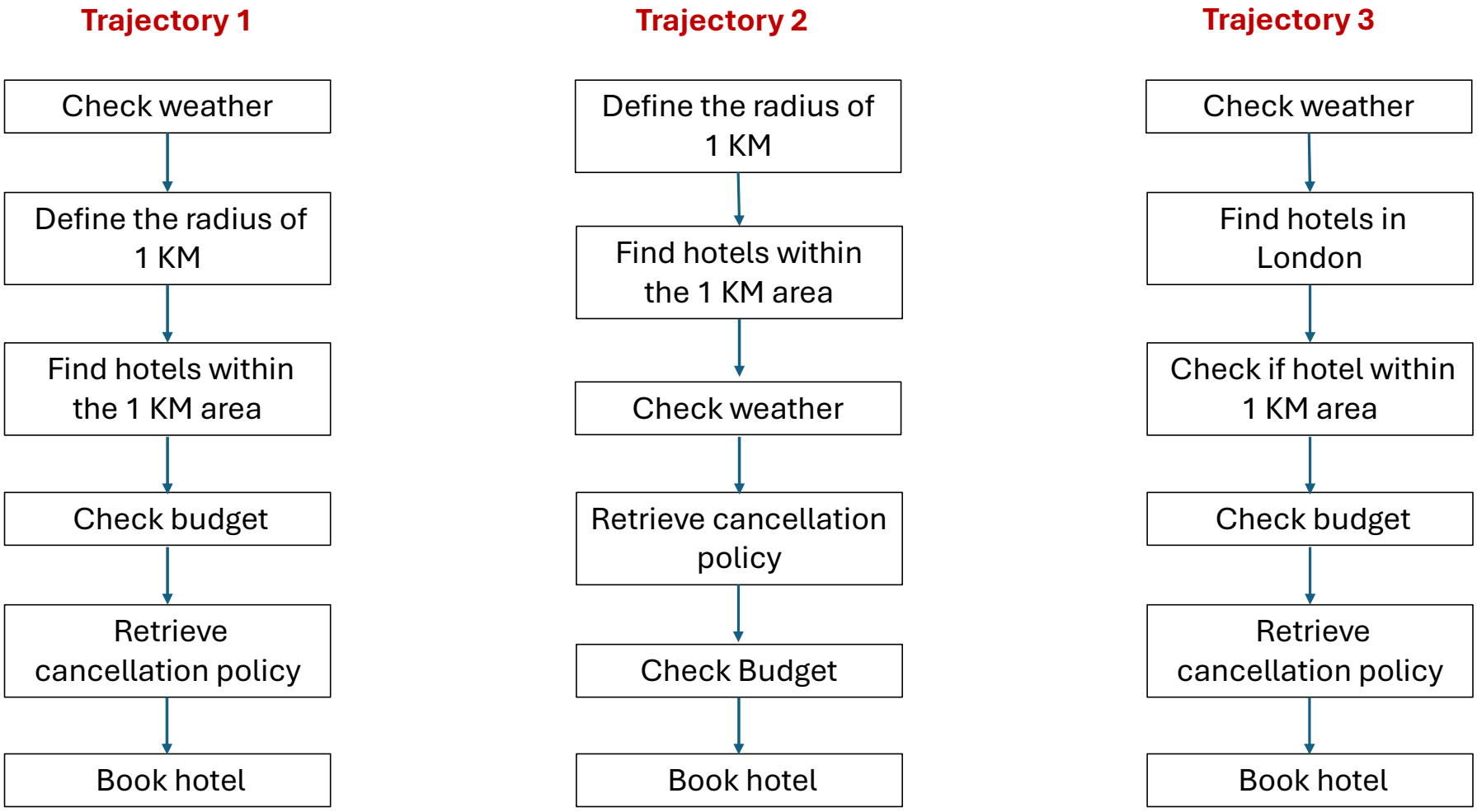


User: Book me a hotel in London for any 2 days in next 2 weeks, when the weather is good. Please consider any hotel less than 200 pounds per night. Make sure the hotel is within 1 KM of Waterloo station. Nearest the better. Also make sure the cancellation policy allows cancellation of the booking

Agent: Hotel ABC has been booked for you from Friday with 175 pounds per night. It is 500 meters from Waterloo station. You can cancel by tomorrow 6 PM. Weather will be good in the entire weekend.

Final Response

- **Task Fulfilment** - Did it fulfil the user's request?
- **Accuracy** - Are its factual claims correct?
- **Groundedness** - Are the claims supported by the available tool or retrieved data?
- **Completeness** - Did it include all required information?
- **Quality** - Is the response relevant and easy to understand?
- **Safety** - Is it safe and compliant?


Trajectory 1
Check weather
Define the radius of 1 KM
Find hotels within the 1 KM area
Check budget
Retrieve cancellation policy
Book hotel
Trajectory 2
Define the radius of 1 KM
Find hotels within the 1 KM area
Check weather
Retrieve cancellation policy
Check Budget
Book hotel
Trajectory 3
Check weather
Find hotels in London
Check if hotel within 1 KM area
Check budget
Retrieve cancellation policy
Book hotel
How do you evaluate the efficiency of the trajectory

- **Exact match** - The same tool calls in the same order as expected

- **In-order match** - All the needed steps in the right order, with extra steps allowed (ex: news tool - check any relevant local news which might be important)

- **Any order match** - All the needed steps, in any order

Precision and Recall

Precision - Of all the tool calls the agent made, how many were actually appropriate? We check weather the agent make unnecessary calls.

Precision Value = Used Correct Tools / Total Tools Used

Recall - Of all the tool calls the task required, how many did the agent actually make used? We check weather the agent miss necessary calls.

Recall Value = Used Required Tools / Total Required Tools

Edge Cases - An edge case test proves it can fail safely

- **Missing information** - Book a hotel for me - ask for more information (place, budget etc)
- **Conflicting instructions** - Book a 5-star hotel with budget less than 200 pound - ask for change in criteria
- **Tool failure, timeout** - Book a hotel for me in London, tool fails to respond - explain something went wrong, do not fabricate information
- **Unauthorized action** - Cancel my manager's hotel booking. - check identity and authorization.
- **Duplicate request** - Book the same hotel twice - should confirm with the user on previous booking
- **Out of scope queries** - Book a cab from hotel - politely define the agent boundary

Golden Dataset

A golden dataset is a set of **reviewed and trusted test scenarios** containing expectations for what the agent should do. mainly used for **repeatable offline regression testing.** Should hold good for every new version of the agent

It should check:

- Did it achieve the expected outcome?
- Was the final response correct and grounded?
- Did it call the required tools?
- Did it make unnecessary calls?
- Did it execute the calls in the expected order?
- Did it work for edge cases

Coverage:

- Happy paths,
- Edge cases
- Safety measures
- Failure scenarios

It should be:

Human-reviewed and versioned

Enhanced overtime from production evaluations

Rubric

How would you judge the quality of response with respect to your criteria?

➢ You define a **rubric** or a *set of clearly defined criteria and scoring rules*

User - Where is my order ORD-101
Agent - Current order status: Shipped. Expected delivery: 30 October.

✓ **Factual accuracy**
- 4: Correct status and delivery date.
- 2: One detail is incorrect or missing.
- 0: Both details are incorrect.

✓ **Completeness**
- 3: Includes both shipment status and expected delivery.
- 1: Includes only one.
- 0: Includes neither.

✓ **Clarity**
- 2: Clear and easy to understand
- 1: Somewhat confusing.
- 0: Unclear.

✓ **Security**
- 1: Only showing details of correct customer
- 0: Showing other customer details

Autoraters

Problem Statement : You define the metrics; you decide the success criteria and judge yourself. It is good for small number of use cases and a small system

Now, when talking about agents, why not a LLM do the judgment

Autoraters: An AI model as the judge. (LLM-as-a-Judge)

A judge model scores each agent response against test criteria's - with a rationale

The autorater takes care:

- The evaluation criteria (rubric)
- The scoring logic
- The prompt/instructions for the judge

Custom Autoraters

Not all your domain judgements can be fulfilled by autoraters,

Use custom autorater

- Bring your own LLM judge
- your instructions
- your criteria + rubric
- Domain-specific rules and business requirements
- Custom judge with your own code -> score

When to do Evaluation?

Offline Evaluation:

- Testing before release
- Predefined test cases
- In the dev or test env
- With test data
- For bug fixing

Online Evaluation:

- Watching quality after release
- Real interaction
- In the prod env
- Real user data
- For Optimization

Demo

Vector & RAG Implementation

Vector Search

- Finds similar items using vector embeddings
- Example – Find books similar to a given book
- Use case – Recommendations, similarity search

RAG Engine

- Retrieves relevant context for LLM responses
- Example – Find relevant books, then explain
- Use case – Q&A chatbots, knowledge assistants

Agent Search

- Fully managed enterprise search solution
- Example – Google-like search across your company library
- Use case – Enterprise search, support portals

Agent retrieval

- Stores and retrieves data using Collections and Data Objects
- Example – Find books using meaning, keywords and filters
- Use case – AI-agent retrieval, semantic and hybrid search

Vector Search

How it works

- You generate embeddings, from any model, and upload them to Cloud Storage
- You create an index and deploy it to an index endpoint
- Queries return the nearest neighbors using approximate nearest neighbor (ANN) search

Used for

- Recommendation engines and semantic search over billions of items
- Chatbots and text classification that rely on semantic matching
- Image matching & Audio search

RAG Engine

How it works

- Imports files from sources such as Cloud Storage and Google Drive
- Embeds and indexes them into a corpus
- Retrieves relevant content at question time and passes it to the LLM

Used for

- Answering questions over your own documents with fewer hallucinations
- Building grounded chatbots and assistants quickly
- Prototyping RAG without wiring every component yourself

Agent Search

How it works

- You create a search engine and connect your data
- Connectors and extensions keep the content fresh
- Embeddings, ranking and grounding are handled for you, with room to customize

Used for

- Search apps over websites, structured and unstructured data
- Enterprise search across apps such as Jira, Confluence and Salesforce
- Out-of-the-box grounding and RAG for generative AI apps

Agent retrieval

How it works

- You work with collections of data objects instead of managing indexes
- Embeddings can be generated automatically and text search is built in
- Self-tuning and fully managed, with Vector Search 1.0 scale and performance

Used for

- One data source for AI apps, with data and embeddings stored together
- Vector, semantic, keyword and hybrid search over the same data
- Agent tools: a remote MCP server lets agents query your collections